

Sistemi Operativi

Nicolò Giuliani

Indice

1	Concorrenza	2
1.1	Introduzione alla concorrenza	2
1.2	Interazioni tra processi	3
1.3	Sezioni critiche	4
1.4	Semafori	8
1.5	Monitor	17
1.6	Message passing	21
1.7	Conclusioni	24
2	Generale	26
2.1	Architettura di un calcolatore	26
2.2	Architettura dei sistemi operativi	29
2.2.1	Struttura del SO	30
2.2.2	Software engineering del SO	31
2.2.3	Macchine virtuali	32
2.2.4	Namespace	32
2.3	Scheduling	32
2.3.1	Scheduler, processi e thread	34
2.3.2	Scheduling	35
2.4	Gestione risorse e deadlock	36
2.4.1	Deadlock	37
2.5	Gestione della memoria	41
2.5.1	Binding, loading, linking	41
2.5.2	Allocazione contigua	42
2.5.3	Paginazione	45
2.5.4	Segmentazione	46
2.5.5	Memoria virtuale	47
2.6	Gestione I/O, Memoria secondaria	50
2.6.1	Progettazione del sistema di I/O	50
2.6.2	Memoria Secondaria	51
2.6.3	RAID	53
2.7		55
3	Linguaggio C per Sistemi Operativi	56
3.1	Unix	56
3.2	Cosa definisce POSIX?	57
3.3	GNU	57

1 Concorrenza

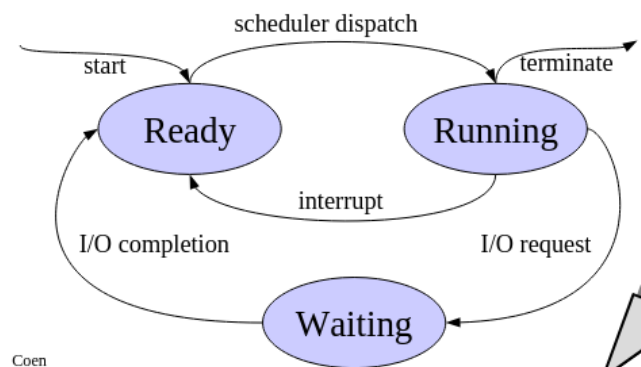
1.1 Introduzione alla concorrenza

Un sistema operativo consiste in un gran numero di attività che vengono eseguite più o meno contemporaneamente dal processore e dai dispositivi presenti in un elaboratore. Ovviamente bisogna far coesistere le diverse attività attraverso il **modello concorrente** e il concetto di **processo**.

Cosa è questo processo? E' una singola attività di un programma che il processore deve eseguire, questo vuol dire che processo $\neq$ programma. Il programma è statico (insieme di istruzioni) mentre il processo è *dinamico*, ovvero l'attività di esecuzione di un programma. Attraverso l'assioma di **finite process** determiniamo che un processo viene eseguito sempre ad una velocità sconosciuta finita, mai nulla.

Come descriviamo un processo?

- Immagine della memoria:
 - la memoria assegnata al singolo processo e le strutture dati del S.O. associate al processo (es. file aperti).
- Immagine del processore:
 - Registri della CPU
- Stato di avanzamento:
 - Running: in esecuzione
 - Waiting: in attesa di evento esterno / non può essere eseguito
 - Ready: può essere eseguito ma la CPU è occupata



Cosa è la concorrenza?

Gestione dei processi *multipli* per la comunicazione e sincronizzazione:

- Multiprogramming: più processi su un singolo processore (parallelismo apparente)
 - i processi sono "alternati nel tempo", quindi 1 sola operazione per istante. Si definisce **interliving**
- Multiprocessing: più processi su più processori (parallelismo reale)
 - i processi sono "alternati nello spazio" ovvero eseguiti simultaneamente su processori diversi. Si definisce **overlapping**
- Distributed processing: più processi su più computer separati (parallelismo reale)

Definiamo in *esecuzione concorrente* due programmi eseguiti in parallelo (apparente/reale). Sia in caso di interliving che per l'overlapping abbiamo gli stessi problemi comuni: predire la velocità dei processi!

Esempio:

	section .data
	.text
void modifica(int valore){	modifica:
totale += valore;	lw \$t0, totale
}	add \$t0, \$t0, \$a0
	sw \$t0, totale
	jr \$ra

Ipotizziamo che eseguiamo P_1 con *modifica*(+10) e P_2 con *modifica*(-10) in esecuzione corrente con la variabile *totale* in comune. Analizziamo l'assembly:

P1 lw \$t0, totale	totale=100, \$t0=100, \$a0=10
P1 add \$t0, \$t0, \$a0	totale=100, \$t0=110, \$a0=10
P1 sw \$t0, totale	totale=110, \$t0=110, \$a0=10

S.O. interrupt

S.O. salvataggio registri P1

S.O. ripristino registri P2 totale=110, \$t0=?, \$a0=-10

P2 lw \$t0, totale	totale=110, \$t0=110, \$a0=-10
P2 add \$t0, \$t0, \$a0	totale=110, \$t0=100, \$a0=-10
P2 sw \$t0, totale	totale=100, \$t0=100, \$a0=-10

Quindi i due processi non possono essere eseguiti contemporaneamente da due processori distinti perchè la variabile *totale* non può essere accesa simultaneamente.

Definiamo **race condition** quando il risultato corretto dipenda dalla temporizzazione dei processi. Per questo esiste lo **scheduler** ovvero il componente del SO che decide il processo da eseguire in determinato momento.

1.2 Interazioni tra processi

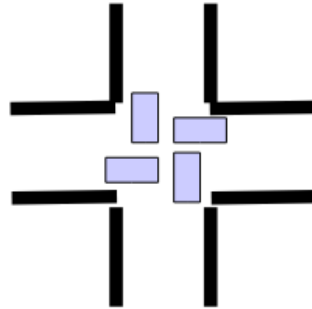
- Processi indirettamente a conoscenza uno dell'altro (memoria pubblica)
 - condividono risorse (buffer), vanno sincronizzati
- Processi direttamente a conoscenza uno dell'altro (memoria privata)
 - la comunicazione viene in base all'id, spesso diretta via messaggi

Una **proprietà** di un programma è un attributo che rimane sempre vero per ogni esecuzione.

- Safety: il programma avanza finchè è tutto ok
 - i processi non devono interferire tra loro nell'accesso della memoria. La sincronizzazione garantisce la proprietà.
- Liveness: il programma è vitale, non si ferma mai. Ha come proprietà la terminazione.
 - i mezzi di sincronizzazione non devono prevenire l'avanzamento del programma (no attese infinite)

Definiamo l'accesso ad una risorsa **mutualmente esclusivo** se per ogni istante solo un processo può accedervi, questo può portare ad un blocco permanente di un programma. Il *deadlock*

ovvero una situazione di stallo dove un processo aspetta un altro creando un attesa a cerchio infinito con mutua esclusione, hold and wait e no uscita forzata, crea una attesa infinita; la sua mancanza è una proprietà di safety.



Quando due processi possono interferire?

Le **Azioni atomiche** sono operazioni che non possono essere interrotte o divise durante la loro esecuzione. Nel parallelismo reale si garantisce che l'azione non interferisca con altri processi durante la sua esecuzione. Nel caso di parallelismo apparante l'avvicendamento fra i processi avviene prima o dopo l'azione, che quindi non può interferire.

```
lw $t0, ($a0)
add $t0, $t0, $a1
sw $t0, ($a0)
```

- le singole istruzioni in linguaggio macchina sono atomiche
- le singole istruzioni in assembly possono non essere atomiche (ci sono psuedoistruzioni)

1.3 Sezioni critiche

Se le sequenze di istruzioni non vengono eseguite in modo atomico, come possiamo garantire la non-interferenza?

Dobbiamo trovare il modo di specificare che certe parti dei programmi sono "speciali", ovvero devono essere eseguite in modo atomico (senza interruzioni).

Definiamo la parte di un programma che utilizza una o più risorse condivise viene detta sezione critica (critical section, o CS)

<pre>process P1 a1 = read(); <u>totale = totale + a1;</u> }</pre>	<pre>process P2 { a2 = read(); <u>totale = totale + a2;</u> }</pre>
---	---

Vogliamo garantire che le sezioni critiche siano eseguite in modo mutualmente esclusivo (atomico), dobbiamo evitare il deadlock e starvation.

<pre> Process P val = rand(); a = a + val; b = b + val Process Q val = rand(); a = a * val; b = b * val; </pre>	<pre> Process P val = rand(); [enter cs] a = a + val; b = b + val [exit cs] Process Q val = rand(); [enter cs] a = a * val; b = b * val; [exit cs] </pre>
---	---

Indichiamo al S.O. cosa può essere eseguito in parallelo e cosa no (in modo atomico).

Ora ragioniamo sul seguente problema:

```

process Pi { /* i=1...N */
  while (true) {
    [enter cs]
    critical section
    [exit cs]
    non-critical section
  }
}

```

dobbiamo avere le seguenti proprietà:

- Mutua esclusione: solo un processo alla volta deve essere all'interno della CS, fra tutti quelli che hanno una CS per la stessa risorsa condivisa
- Assenza di deadlock: uno scenario in cui tutti i processi restano bloccati definitivamente non è ammissibile
- Assenza di delay non necessari: un processo fuori dalla CS non deve ritardare l'ingresso della CS da parte di un altro processo
- Eventual entry (assenza di starvation): ogni processo che lo richiede, prima o poi entra nella CS

- Algoritmo di Peterson:

```

shared boolean needp = false;
shared boolean needq = false;
shared int turn;

```

```

process P {
    while (true) {
        /* entry protocol */
        needp = true;
        turn = Q;
        while (needq && turn != P) ;
        /* do nothing */
        critical section
        needp = false;
        non-critical section
    }
}

```

```

process Q {
    while (true) {
        /* entry protocol */
        needq = true;
        turn = P;
        while (needp && turn != Q) ;
        /* do nothing */
        critical section
        needq = false;
        non-critical section
    }
}

```

Vediamo la versione per n processi

```

shared int [] stage = new int [N]; /* 0-initialized */
shared int [] last = new int [N]; /* 0-initialized */
process Pi { /* i = 0...N-1 */
    while (true) {
        /* Entry protocol */
        for (int j=0; j < N; j++) {
            stage[i] = j; last[j] = i;
            for (int k=0; k < N; k++) {
                if (i != k)
                    while (stage[k] >= stage[i] && last[j] == i)
                        ;
            }
        }
        critical section
        stage[i] = 0;
        non-critical section
    }
}

```

A livello teorico, è possibile creare istruzioni hardware speciali per semplificare la realizzazione delle sezioni critiche. Tali istruzioni eseguono due azioni in modo atomico, e vengono spesso

utilizzate per implementare meccanismi di *spinlock*.

$$TS(x, y) := \langle y = x; x = 1 \rangle$$

Ciò significa che, con l'operazione di *Test* (controlla/legge il valore) e *Set* (imposta/scrive), si legge se la risorsa condivisa x è libera (0) o occupata (1), e si assegna alla variabile locale y il valore precedente di x . L'intera operazione avviene in modo atomico.

```
shared int lock = 0;    // 0 = libero , 1 = occupato

process P {
    int vp;
    while (true) {
        do {
            TS(lock , vp);    // vp = lock; lock = 1 (atomico)
        } while (vp);        // se vp = 1 -> lock occupato , riprova

        critical_section();    // sezione critica (solo uno per volta)
        lock = 0;              // rilascio del lock
        non_critical_section();
    }
}

process Q {
    int vp;
    while (true) {
        do {
            TS(lock , vp);
        } while (vp);

        critical_section();
        lock = 0;
        non_critical_section();
    }
}
```

1.4 Semafori

Si tratta di un paradigma per la sincronizzazione (così come i semafori stradali sincronizzano l'occupazione di un incrocio). Vediamo ora il principio alla base: due o più processi possono cooperare attraverso semplici segnali, in modo tale che un processo possa essere bloccato in specifici punti del suo programma finché non riceve un segnale da un altro processo.

```
Semaphore s = new Semaphore(1);
process P {
    while (true) {
        s.P();
        critical section
        s.V();
        non-critical section
    }
}
```

Analizziamo:

- ogni semaforo è una variabile intera, inizializzato a valore **non negativo**
- P (atomico): attende che il semaforo sia *positivo*, decrementa il semaforo
- V (atomico): incrementa il semaforo

```
class Semaphore {
    semaphore(int v);
    void P(void);
    void V(void);
}
```

- n_p è il numero di operazioni di P
- n_v è il numero di operazioni di V
- $init$ è il valore iniziale del semaforo

Il valore del semaforo: $n_v + init - n_p$, quindi
 $n_p \leq n_v + init$

Semafori – Implementazione nei sistemi operativi

```
void P() {
    [enter CS]
    if (value > 0)
        value--;
    else {
        int pid = <id del processo
        che ha invocato P>;
        queue.enqueue(pid);
        suspend(pid);
    }
    [exit CS]
}

void V() {
    [enter CS]
    if (queue.empty())
        value++;
    else {
        int pid = queue.dequeue();
        wakeup(pid);
    }
    [exit CS]
}
```

Utilizzando queste tecniche, non abbiamo eliminato busy-waiting, però lo abbiamo limitato alle sezioni critiche di P e V , e queste sezioni critiche sono molto brevi.


```
<enter CS>  
/*codice critico*/  
  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
. . .  
  
/*fine cod.crit.*/  
<exit CS>
```

*potenziale
busy waiting*

[illegible]

Variante dei semafori in cui il valore può assumere solo i valori 0 e 1.

$$0 \leq s.value \leq 1$$

```
class BinarySemaphore {
    private int value;
    Queue queue0 = new Queue();
    Queue queue1 = new Queue();
    BinarySemaphore() { value = 1;}
}
```

}

}

Possiamo definire i semafori generali attraverso semafori binari

```
class Semaphore {
    private BinarySemaphore mutex(1);
    int value;
    QueueOfBinSem queue = new QueueOfBinSem();
    Semaphore(int v) { // +fail if v < 0
        value = v;}
}
```

- un semaforo **mutex** per garantire mutua esclusione sulle variabili
- un semaforo privato creato in allocazione dinamica per ogni processo che partecipa
- una coda per garantire fairness

```
void P() {
    mutex.P();
    if (value > 0) {
        value--;
        mutex.V();
    } else {
        S = new BinarySemaphore(0);
        queue.enqueue(S);
        mutex.V();
        S.P();
        free(S);
    }
}

void V() {
    mutex.P();
    if (queue.empty())
        value++;
    else {
        BinarySemaphore s = queue.dequeue();
        s.V();
    }
    mutex.V();
}
```

```
class BinarySemaphore {
    private Semaphore s0, s1;
    int value;
    BinarySemaphore(int v) { // +fail if v not in {0,1}
        S0 = new semaphore(v)
        S1 = new semaphore(1-v)
    }
    void P(void) {
        S0.P();
        S1.V();
    }
    void V(void) {
        S1.P();
        S0.V();
    }
}
```

Produttore/Consumatore

Esiste un processo "produttore" Producer che genera valori (record, caratteri, oggetti, etc.) e vuole trasferirli a un processo "consumatore" Consumer che prende i valori generati e li "consuma". La comunicazioni avviene attraverso una variabile condivisa.

```

shared Object buffer;
Semaphore empty =
    new Semaphore(1);
Semaphore full =
    new Semaphore(0);

process Producer {
    while (true) {
        Object val = produce();
        empty.P();
        buffer = val;
        full.V();
    }
}

process Consumer {
    while (true) {
        full.P();
        Object val = buffer;
        empty.V();
        consume(val);
    }
}

```

Buffer limitato

In questo caso, però, lo scambio tra produttore e consumatore non avviene tramite un singolo elemento, ma tramite un buffer di dimensione limitata, i.e. un vettore di elementi.

```

Queue q(maxsize = SIZE);
Semaphore empty =
    new Semaphore(SIZE);
Semaphore full =
    new Semaphore(0);
Semaphore mutex =
    new Semaphore(1);

process Producer {
    while (true) {
        Object val = produce();
        empty.P();
        mutex.P();
        q.enqueue(val);
        mutex.V();
        full.V();
    }
}

process Consumer {
    while (true) {
        Object val;
        full.P();
        mutex.P();
        val = q.dequeue();
        mutex.V();
        empty.V();
        consume(val);
    }
}

```

È possibile utilizzare il codice precedente con produttori e consumatori multipli?

Il **problema dei filosofi a cena** è un classico esempio di *sincronizzazione concorrente*:

- Ci sono 5 filosofi seduti attorno a un tavolo circolare.
- Tra ogni coppia di filosofi c'è una bacchetta.
- Ogni filosofo alterna due attività: **pensare** ed **mangiare**.
- Per mangiare, un filosofo ha bisogno **di entrambe le bacchette adiacenti**.
- Se tutti i filosofi prendono contemporaneamente la bacchetta alla loro destra (o sinistra), nessuno potrà mai prendere la seconda bacchetta, creando un **deadlock** (attesa circolare infinita).

```
Semaphore chopsticks = { new Semaphore(1), ..., new Semaphore(1) };
```

```
process Philo[i] { /* i = 0...3 */
  while (true) {
    think
    chopstick[i].P();
    chopstick[i+1].P();
    eat
    chopstick[i].V();
    chopstick[i+1].V();
  }
}

process Philo[4] {
  while (true) {
    think
    chopstick[0].P();
    chopstick[4].P();
    eat
    chopstick[0].V();
    chopstick[4].V();
  }
}
```

La soluzione si basa su due concetti chiave:

1. **Eliminare l'attesa circolare:** rompere la simmetria nella sequenza di presa delle bacchette, evitando che tutti seguano lo stesso ordine.
2. **Rompere la simmetria con un filosofo “mancino”:** tutti i filosofi prendono prima la bacchetta alla loro destra e poi quella alla sinistra, **tranne uno** che fa il contrario. Questo garantisce che almeno un filosofo possa mangiare, liberando le bacchette per gli altri.

Implementazione con semafori

- Ogni bacchetta è rappresentata da un **semaforo binario** inizializzato a 1.
- I filosofi “destri” prendono prima la bacchetta sinistra, poi quella destra.
- Il filosofo “mancino” prende prima la bacchetta destra, poi quella sinistra.
- Dopo aver mangiato, ciascun filosofo rilascia entrambe le bacchette.

Vantaggio: questa semplice modifica evita il deadlock pur mantenendo il codice semplice.

Lettori e scrittori

Un database è condiviso tra un certo numero di processi di due tipologie

- lettori: accedono al database per leggerne il contenuto
- scrittori: accedono al database per aggiornarne il contenuto

Se uno scrittore sta scrivendo il database è in mutua esclusione, mentre per la lettura abbiamo più lettori contemporaneamente.

Questo avviene attraverso **classi di processi** così la mutua esclusione e la condivisione possono coesistere.

```
process Reader {
    while (true) {
        startRead();
        read the database
        endRead();
    }
}

process Writer {
    while (true) {
        startWrite();
        write the database
        endWrite();
    }
}
```

Quindi un lettore potrà leggere in qualsiasi momento tranne che se uno scrittore stia scrivendo, mentre uno scrittore deve aspettare che i lettori finiscano di leggere (minima attesa possibile)

```
/* Variabili condivise */
int nr = 0;
Semaphore rw = new Semaphore(1);
Semaphore mutex = new Semaphore(1);

void startRead() {
    mutex.P();
    if (nr == 0)
        rw.P();
    nr++;
    mutex.V();
}

void startWrite() {
    rw.P();
}

void endRead() {
    mutex.P();
    nr--;
    if (nr == 0)
        rw.V();
    mutex.V();
}

void endWrite() {
    rw.V();
}
```

Questa soluzione però limita molto i lettori. Come possiamo risolverla?

Introduciamo una notazione prima di arrivare alla soluzione finale:

- **B** condizione booleana
- **S** uno statement

- $\langle S \rangle$: esegui lo statement S in modo atomico
- $\langle \text{await}(B) \rightarrow S \text{ attendi (atomico) finchè } B \text{ non si vero e quindi esegui (atomico) } S \rangle$
Quindi quando S è eseguito B è verificata

Quindi la logica sarebbe

```
< S >
  mutex.P();
  S;
  SIGNAL();

< await(Bi) -> Si >
  mutex.P();
  if (!Bi) {
    waiting[i]++;
    mutex.V();
    sem[i].P();
    waiting[i]--;
  }
  Si;
  SIGNAL();
```

```
void SIGNAL() //non deterministica
{
  if (B0 && waiting[0]>0)
    sem[0].V();
  if (B1 && waiting[1]>0)
    sem[1].V();
  ...
  if (Bn-1 && waiting[n-1]>0)
    sem[n-1].V();
  if (!(B0 && waiting[0]>0) && !(B1 && waiting[1]>0) && ...
    !(Bn-1 && waiting[n-1]>0) )
    mutex.V();
}
```

Questa trasformazione si chiama passaggio del testimone ("*passing the baton*"). Ovvero SIGNAL verifica se esiste un processo, fra quelli in attesa, che possono proseguire se esiste, "gli passa il testimone" (la mutua esclusione) altrimenti la rilascia. Si definisce come un semaforo binario suddiviso tra i vari semafori (split binary semaphore).

```
/* Introduced by transformation */
Semaphore mutex = new Semaphore(1); /* Mutual exclusion */
Semaphore semr = new Semaphore(0); /* Reader semaphore */
Semaphore semw = new Semaphore(0); /* Writer semaphore */
int waitingr = 0; /* Number of waiting reader */
int waitingw = 0; /* Number of waiting writer */
/* Problem variables */
int nr = 0; /* Number of current readers */
int nw = 0; /* Number of current writers */
```

```

process Reader {
    while (true) {
        mutex.P();
        if (nw > 0) {
            waitingr++;
            mutex.V();
            semr.P();
            waitingr--;
        }
        nr++;
        SIGNAL();
        read the database
        mutex.P();
        nr--;
        SIGNAL();
    }
}

process Writer {
    while (true) {
        mutex.P();
        if (nr > 0 || nw > 0) {
            waitingw++;
            mutex.V();
            semw.P();
            waitingw--;
        }
        nw++;
        SIGNAL();
        write the database
        mutex.P();
        nw--;
        SIGNAL();
    }
}

void SIGNAL() {
    if ( (nw == 0) && waitingr > 0)
        semr.V();
    if ( (nw == 0 && nr == 0) && waitingw > 0)
        semw.V();
    if ( !( (nw == 0) && waitingr > 0 ) &&
        !( (nw == 0 && nr == 0) && waitingw > 0 ) ){
        mutex.V()
    }
}

```

Vediamo ora: R/W trasformato (SIGNAL ridotto, non-determinismo eliminato)

```

process Reader {
    while (true) {
        mutex.P();
        if (nw > 0) {
            waitingr++;
            mutex.V();
            semr.P();
            Waitingr--;
        }
        nr++;
        if (waitingr > 0)
            semr.V();
        else
            mutex.V();
        read the database
        mutex.P();
        nr--;
        if (nr == 0 && waitingw > 0)
            semw.V();
        else
            mutex.V();
    }
}

process Writer {
    while (true) {
        mutex.P();
        if (nr > 0 || nw > 0) {
            waitingw++;
            mutex.V();
            semw.P();
            Waitingw--;
        }
        Nw++;
        mutex.V();
        write the database
        mutex.P();
        nw--;
        if (waitingr > 0)
            semr.V();
        else if (waitingw > 0)
            semw.V();
        else
            mutex.V();
    }
}

```

Queste versioni danno priorità ai lettori e quindi abbiamo starvation per gli scrittori. Vediamo ora la versione senza la starvation

```

process Reader {
    while (true) {
        mutex.P();
        if (nw > 0 || waitingw > 0){
            waitingr++; mutex.V();
            semr.P(); waitingr--;
        }
        nr++;
        if (waitingr > 0)
            semr.V();
        else
            mutex.V();
        read the database
        mutex.P();
        nr--;
        if (nr == 0 && waitingw > 0)
            semw.V();
        else
            mutex.V();
    }
}

process Writer {
    while (true) {
        mutex.P();
        if (nr > 0 || nw > 0) {
            waitingw++; mutex.V();
            semw.P(); waitingw--;
        }
        nw++;
        mutex.V();
        write the database
        mutex.P();
        nw--;
        if (waitingr > 0)
            semr.V();
        else if (waitingw > 0)
            semw.V();
        else
            mutex.V();
    }
}

```

Il barbiere addormentato

Un negozio di barbiere ha un barbiere, una poltrona da barbiere e n sedie per i clienti in attesa. Quando non ci sono clienti il barbiere si mette sulla sedia da barbiere e si addormenta.

Quando arriva un cliente, sveglia il barbiere addormentato e si fa tagliare i capelli sulla sedia da barbiere. Se arriva un cliente mentre il barbiere sta tagliando i capelli a un altro cliente, il cliente si mette in attesa su una delle sedie. Se tutte le sedie sono occupate, il cliente se ne va scocciato!

Da come abbiamo visto i semafori hanno diverse falle, a partire dagli errori "banali" del programmatore alla leggibilità scarsa del codice. Che alternative abbiamo?

1.5 Monitor

Un monitor è un modulo software che consiste di:

- dati locali
- sequenza di inizializzazione

Le caratteristiche sono

- i dati locali sono accessibili solo alle procedure del modulo stesso
- un processo entra in un monitor invocando una delle sue procedure
- solo un processo alla volta può essere all'interno del monitor, gli altri processi che invocano il monitor sono sospesi

Vediamo una "pseudo-sintassi" della struttura

```
monitor name {  
    variable declarations ...  
    procedure entry type procedurename1(args ...) {  
        ...  
    }  
    type procedurename2(args ...) {  
        ...  
    }  
    name(args ...) {  
        ...  
    }  
}
```

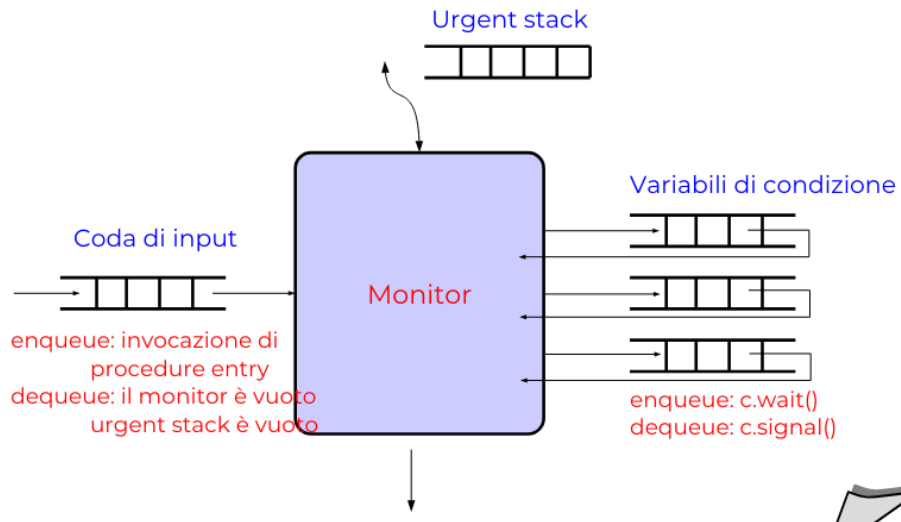
A livello tecnico la programmazione ad oggetti ci semplifica la vita infatti la inizializzazione coincide con il costruttore

- `Inizializzaione` = Costruttore
- `Procedure entry` = metodi pubblici dell'oggetto
- `Procedure normali` = metodi privati
- `Variabili locali` = variabili private

Da come abbiamo detto solo un processo alla volta può essere all'interno del monitor, quindi questo ci fornisce un semplice strumento di mutua esclusione.

Come logica definiamo

- condition c;
- c.wait()
attende il verificarsi della condizione, viene sospeso in una coda di attesa della condizione c
- c.signal()
segnala che la condizione è vera, causa la riattivazione immediata di un processo



Signal urgent (SU) è la politica "classica" di signaling.
Possiamo anche implementare i **semafori** con questa logica.

```
monitor Semaphore {
  int value;
  condition c; /* value > 0 */
  procedure entry void P() {
    if (value == 0)
      c.wait();
    value--;
  }
  procedure entry void V() {
    value++;
    c.signal();
  }
  Semaphore(int init) {
    value = init;
  }
}
```

E per il **R/W**? Vediamo la versione senza la starvation

```
monitor RWController
  int nr; /* number of readers */
  int nw; /* number of writers */
  condition okToRead; /* nw == 0 */
  condition okToWrite; /* nr == 0 && nw == 0 */

  procedure entry void startRead() {
```

```

    if (nw > 0 || ww > 0) okToRead.wait();
    nr = nr + 1;
    okToRead.signal();
}
procedure entry void endRead() {
    nr = nr - 1;
    if (nr == 0) okToWrite.signal();
}
procedure entry void startWrite() {
    if (nr > 0 || nw > 0) { ww++; okToWrite.wait(); ww--; }
    nw = nw + 1;
}
procedure entry void endWrite() {
    nw = nw - 1;
    okToRead.signal();
    if (nr == 0) okToWrite.signal();
}

```

Invece per **Produttore / Consumatore**? Abbiamo anche quella

```

monitor PCController {
    Object buffer;
    condition empty;
    condition full;
    boolean isFull;
    PCController() {
        isFull=false;
    }

    procedure entry Object read() {
        if (!isFull)
            full.wait();
        int retvalue = buffer;
        isFull = false;
        empty.signal();
        return retvalue;
    }

    procedure entry void write(int val){
        if (isFull)
            empty.wait();
        buffer = val;
        isFull = true;
        full.signal();
    }
}

```

```

process Producer {
    Object x;
    while (true) {
        x = produce();
        pcController.write(x);
    }
}

process Consumer {
    Object x;
    while (true) {
        x = pcController.read();
        consume(x);
    }
}

```

Buffer limitato tramite Monitor

```

monitor PCController {
    QueueOfObj q;
    int maxsz
    condition okRead; // q.size > 0.
    condition okWrite; // q.size < maxsz
    PCController(int size) {
        maxsz = size;
    }

    procedure entry void write(Obj val)
    {
        if (q.size() >= maxsz)
            okWrite.wait();
        q.enqueue(val);
        okRead.signal();
    }

    procedure entry Object read() {
        if (q.size() == 0)
            okRead.wait();
        Obj retval = q.dequeue();
        okWrite.signal();
        return retval;
    }
}

```

Filosofi a cena

```

monitor DPController {
    condition unusedchopstick[5];
    boolean chopstick[5];
    procedure entry void startEating(int i) {
        if (chopstick[i])
            unusedchopstick[i].wait();
        chopstick[i] = true;
        if (chopstick[(i+1)%5])
            unusedchopstick[(i+1)%5].wait();
        chopstick[(i+1)%5] = true;
    }
    procedure entry void finishEating(int i) {
        chopstick[i] = false;
        chopstick[(i+1)%5] = false;
        unusedchopstick[i].signal();
        unusedchopstick[(i+1)%5].signal();
    }
}

monitor chopstick[i] {

```

```

boolean inuse = false;
condition free;
procedure entry void pickup() {
    if (inuse)
        free.wait();
    inuse = true;
}
procedure entry void putdown() {
    inuse = false;
    free.signal();
}
}

process Philo[i] {
    while (true) {
        think
        chopstick[MIN(i, (i+1)%5)].pickup();
        chopstick[MAX(i, (i+1)%5)].pickup();
        eat
        chopstick[MIN(i, (i+1)%5)].putdown();
        chopstick[MAX(i, (i+1)%5)].putdown();
    }
}

```

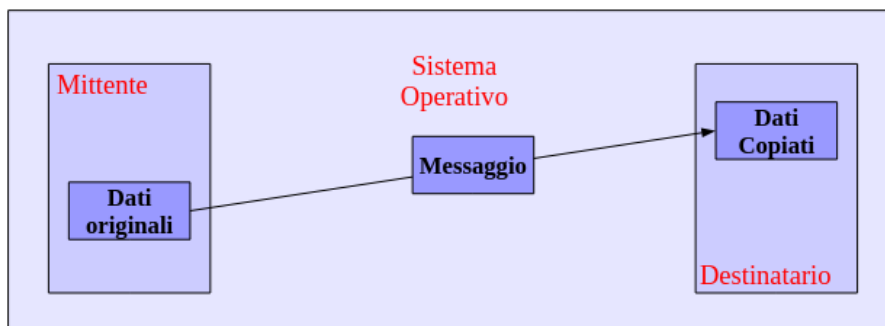
Ma come li implementiamo i monitor?

- Uno stack (per urgent)
- Una queue (per waiting queue)
- Un semaforo

E poi il codice della struttura e relative funzioni.

1.6 Message passing

Fino ad ora abbiamo visto metodi per sincronizzare i processi basati sulla memoria, ora ragioniamo con lo scambio di messaggi, e non più semplici segnali.

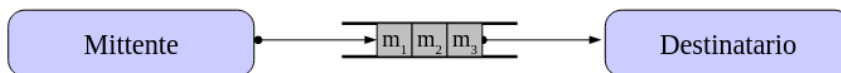


- send: "spedire" un messaggio ad un processo destinatario
- receive: "ricevere" un messaggio da un processo mittente

Abbiamo 3 tipologie di message passing

- MP sincrono
 - send sincrona: il mittente spedisce il messaggio al processo, restando bloccato fino a quando il destinatario non riceve il messaggio
 - receive bloccante: il destinatario riceve un messaggio dal processo send. Se il mittente non ha ancora spedito alcun messaggio, il destinatario si blocca in attesa di ricevere un messaggio
- MP asincrono
 - send asincrono: il mittente spedisce il messaggio al processo destinatario, senza bloccarsi in attesa che il destinatario riceva il messaggio
 - receive bloccante: il destinatario riceve un messaggio dal processo send. Se il mittente non ha ancora spedito alcun messaggio, il destinatario si blocca in attesa di ricevere un messaggio
- MP completamente asincrono
 - send asincrona: il mittente spedisce il messaggio al processo destinatario, senza bloccarsi in attesa che il destinatario riceva il messaggio
 - receive non bloccante: il destinatario riceve un messaggio dal processo send. Se il mittente non ha ancora spedito alcun messaggio, la nb-receive (la sua funzione) termina ritornando un messaggio "nullo"

Message passing asincrono



Message passing sincrono



Noi utilizzeremo un message passing **asincrono** con **send**, **receive**, **reply**. Vediamo quindi un MP asincrono dato quello sincrono (senza gestione ANY)

```

/* p is the calling process */
void asend(msg_t m, pid_t dst) {
    ssend("SND(m,getpid(),dst)", server);
}
msg_t areceive(pid_t snd) {
    ssend("RCV(getpid(),snd)", server);
    msg_t m = sreceive(server);
    return m;
}
  
```

```

}
process server {
    /* One element x process pair */
    int [][] waiting;
    Queue [][] queue;
    while (true) {
        handleMessage();
    }
}

void handleMessage() {
    msg = sreceive(0);
    if (msg == <SND(m,p,q)>) {
        if (waiting[p,q]>0) {
            ssend(m, q);
            waiting[p,q]--;
        } else {
            queue[p,q].add(m);
        }
    } else if (msg == <RCV(q,p)>) {
        if (queue[p,q].isEmpty()) {
            waiting[p,q]++;
        } else {
            m = queue[p,q].remove();
            ssend(m, q);
        }
    }
}
}

```

Message Passing - Filosofi a cena

```

process Philo[i] {
    while (true) {
        think
        asend(<PICKUP,i>, chopstick[MIN(i, (i+1)%5)]);
        msg = areceive(chopstick[MIN(i, (i+1)%5)]);
        asend(<PICKUP,i>, chopstick[MAX(i, (i+1)%5)]);
        msg = areceive(chopstick[MAX(i, (i+1)%5)]);
        eat
        asend(<PUTDOWN,i>, chopstick[MIN(i, (i+1)%5)]);
        asend(<PUTDOWN,i>, chopstick[MAX(i, (i+1)%5)]);
    }
}

process chopstick[i] {
    boolean free = true;
    Queue queue = new Queue();
    while (true) {
        handleRequests();
    }
}

void handleRequests() {

```

```

msg = areceive(0);
if (msg == <PICKUP,j>) {
    if (free) {
        free = false;
        asend(ACK, philo[j]);
    } else {
        queue.add(j);
    }
}
else if (msg == <PUTDOWN, j>) {
    if (queue.isEmpty()) {
        free = true;
    } else {
        k = queue.remove();
        asend(ACK, philo[k]);
    }
}
}
}

```

Message Passing - Produttori e consumatori

```

process PCmanager {
    Object x;
    while (true) {
        x = sreceive(Producer);
        ssend(x, Consumer);
    }
}

```

```

process Consumer{
    Object x;
    while (true) {
        x = sreceive(PCmanager);
        consume(x);
    }
}

```

```

process PCmanager {
    Object x;
    while (true) {
        x = sreceive(Producer);
        ssend(x, Consumer);
    }
}

```

1.7 Conclusioni

- Sezioni critiche: realizzare mutua esclusione in sistemi mono e multiprocessore all'interno del sistema operativo stesso. (troppo basso livello)
- Semafori: primitiva di sincronizzazione, effettivamente offerta dai S.O. (troppo basso)
- Monitor: meccanismi integrati nei linguaggi di programmazione
- Message passing: meccanismo più diffuso, può essere poco efficiente

Si possono tracciare le seguenti classi di paradigmi:

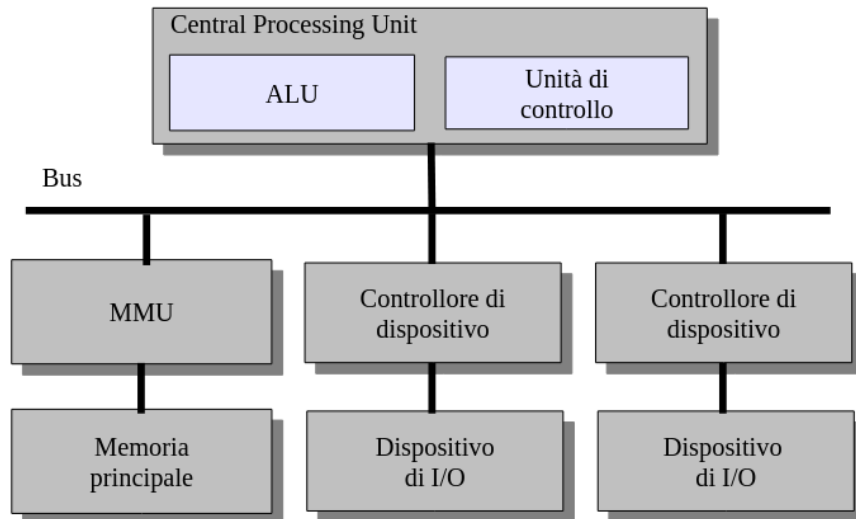
- Metodi a memoria condivisa:
 - semafori, semafori binari, monitor (stesso potere espressivo)
 - dekker e peterson, Test&Set necessitano di busy waiting
- Metodi a memoria privata:
 - message passing asincrono ha maggiore potere espressivo
 - message passing sincrono

Ricorda: si dice che il paradigma di programmazione A è espressivo almeno quanto il paradigma di programmazione B (e si scrive $A \geq B$) quando è possibile esprimere ogni programma scritto con B mediante A . Ovvero, quando è possibile scrivere una libreria che consenta di implementare le chiamate di un paradigma B esprimendole in termini di A si avrà $A \geq B$.

2 Generale

2.1 Architettura di un calcolatore

Ripassiamo la architettura di Von Neumann



Definiamo

- Device: hardware
- Device controller: scheda elettronica che controlla il dispositivo (hardware)
- Device driver: software
- Memory management unit: unità controllo memoria (hardware)
- Memory manager: parte del S.O. che gestisce la MMU (software)

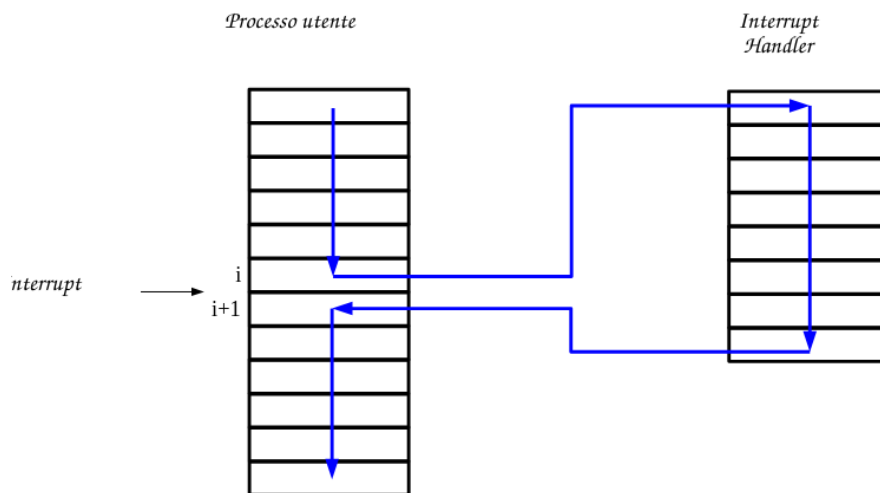
Interrupt Meccanismo che interrompe il ciclo della CPU. Permettono al S.O. di intervenire durante l'esecuzione di un programma.

- Interrupt hardware: eventi asincroni, non causati dal processo in esecuzione. (I/O, interval time)
- Interrupt software (trap): causato dal programma. (divisione per 0, chiamate di system call)

Gli interrupt sono inviati al/ai processori, o alle schede controller del dispositivo.

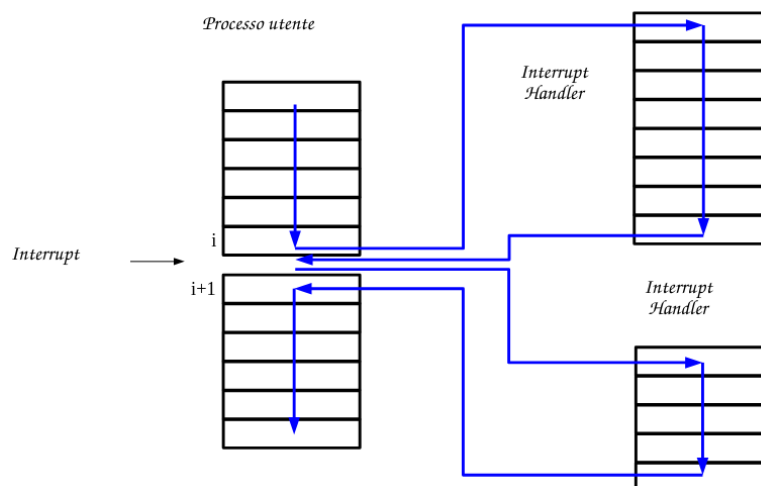
Cosa succedere dopo aver chiamato un interrupt?

- 1) Invio di *interrupt request* al processore
- 2) Il processore sospende le operazioni, salta ad un indirizzo specifico (interrupt handler)
- 3) Qua nel interrupt handler gestiamo correttamente l'interrupt, ritorniamo il controllo al processo interrotto (o ad un altro processo)
- 4) Il processore riprende l'esecuzione, come se non fosse successo niente

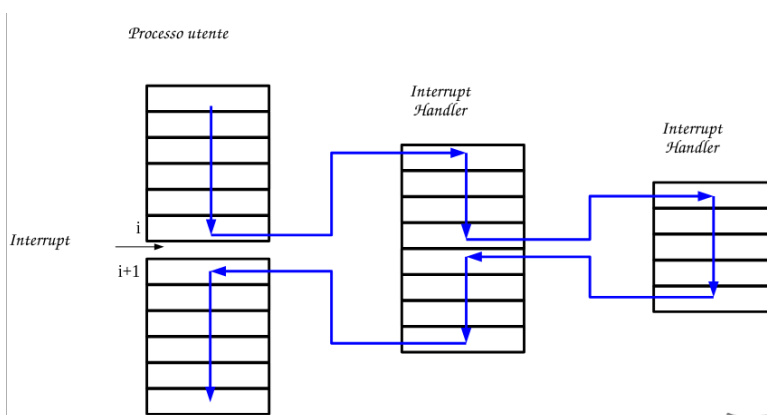


E nel caso se avessimo più interrupt?

- Disabilitazione degli interrupt: se durante l'esecuzione di un interrupt handler riceviamo un altro interrupt lo ignoriamo fino ad avere finito il nostro interrupt handler per poi eseguirlo dopo.



- Interrupt annidati: Abbiamo delle priorità, e uno inferiore può essere interrotto da uno di priorità maggiore.



Comunicazione fra processore e dispositivi di I/O Il controller fa da tramite tra il dispositivo I/O e il processore. Come lo implementiamo?

- **Programmed I/O (obsoleto):** La CPU carica nel controller la richiesta, il controller esegue e si copia in un suo buffer il risultato. Nel frattempo il SO controlla quando l'operazione è stata completata. Si segna in appositi suoi registri del controller che l'operazione è stata completata. La CPU copia i dati nella memoria.
- **Interrupt-Driven I/O:** La CPU carica nel controller la richiesta, il SO sospende il processo e nel frattempo ne esegue un altro. Il controller nel frattempo esegue, copia nel suo buffer e segnala alla CPU via un interrupt il completamento. La CPU copia i dati dal buffer alla memoria.

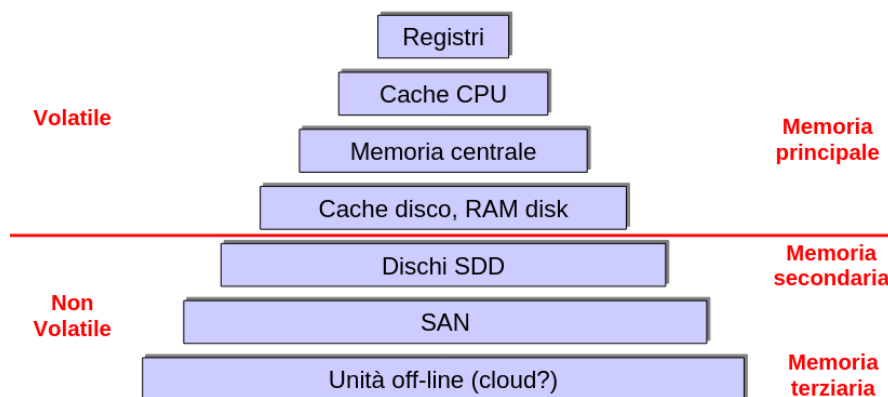
Direct Memory Access (DMA) Il controller del dispositivo legge/scrive i dati direttamente dalla memoria centrale, senza passare dalla CPU. Il SO specifica l'indirizzo di memoria.

Memoria Insieme ai registri è l'unico spazio di memoria che il processore può accedervi direttamente, nei sistemi moderni abbiamo anche la MMU che permette l'accesso modificando gli indirizzi logici in indirizzi fisici.

Memory Mapped I/O Tramite il bus di sistema mappiamo i dati del dispositivo tramite indirizzi direttamente accessibili. La lettura/scrittura sul bus comporta il trasferimento di dati da o verso il dispositivo. Tutto questo necessita tecniche di sincronizzazione di accesso.

Dischi Consentono la memorizzazione non volatile dei dati. Accesso random diretto. Attraverso il controller possiamo:

- READ(head, sector)
- WRITE(head, sector)
- SEEK(cylinder): spostamento fisico della testina da un cilindro all'altro.



Cache Memorizzare parzialmente i dati di una memoria in una seconda più costosa ma più efficiente. Il caching lo troviamo nella DRAM attraverso la memoria bipolare (L1, L2), cache su disco fisso (cache HW), cache di file system remoti tramite file sistem locali (cache SW). Bisogna però avere il giusto algoritmo di caching che permetta di garantire il maggior numero possibile di accessi.

Nei sistemi multiprogramma e multiutente dobbiamo avere meccanismi di protezione per evitare che i processi concorrenti creino conflitto. Li creiamo via HW o SW?

- Hardware:
 - Kernel: i processi qui hanno tutte le istruzioni, incluse istruzioni privilegiate, per gestire il sistema
 - Utente: i processi in modalità utente non hanno accesso alle istruzioni privilegiate

Come funziona?

- CPU modalità Kernel
- Caricamento del SO (bootstrap), ed sua esecuzione
- Quando abbiamo un processo utente cambiamo il valore del *mode bit* (nel registro) e mettiamo la CPU modalità utente
- Quando riceviamo un interrupt passiamo a modalità kernel

Per garantire anche la protezione della memoria utilizziamo il MMU, nel quale l'indirizzo logico viene tradotto in indirizzo fisico e nel caso di traduzione impossibile l'indirizzo viene protetto.

2.2 Architettura dei sistemi operativi

Gestione dei processi

Un processo utilizza le risorse fornite dal computer per svolgere i propri compiti. Il SO si deve occupare di creazione/terminazione dei processi, sospensione/riattivazione, gestione deadlock, comunicazione fra i processi, sincronizzazione dei processi.

Gestione della memoria principale

La RAM è un array di dati accessibile dalla CPU e controller I/O se DMA. Il SO deve tenere traccia della memoria utilizzata e da chi, allocare/deallocare, decidere i processi da caricare, usare memoria secondaria per amplificare la memoria primaria.

Gestione della memoria secondaria

Il SO gestisce il partizionamento, RAID, ordinamento efficiente delle richieste (disk scheduling).

Gestione file system

Un file system è composto da un insieme di file. Il SO si occupa della creazione/cancellazione dei file e directory, manipolazione file/directory, codifica del file system su una sequenza di blocchi.

Gestione dei dispositivi di I/O

Driver per hw specifico, gestione del buffer delle informazioni e una interfaccia comune per gestione dei driver.

Supporto multiuser

Abbiamo bisogno di un meccanismo di protezione per controllare gli accessi dei processi alle risorse del sistema e degli utenti. Il SO deve garantire la protezione come: garantire identità de proprietario del processo, gestire chi può fare cosa.

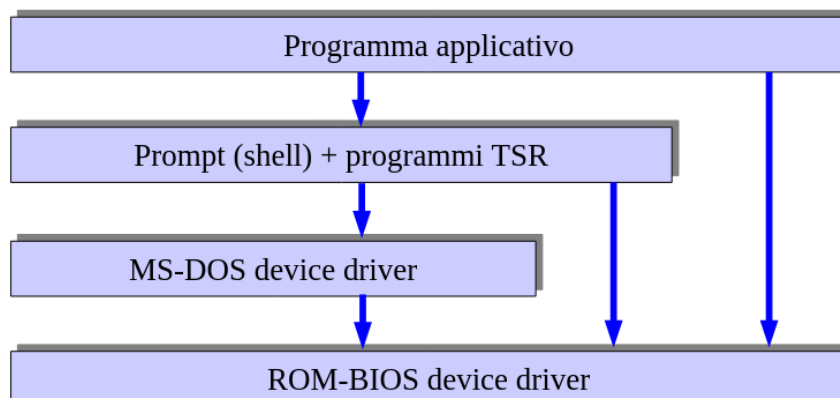
Networking

Consente di far comunicare processi in esecuzione su più elaboratori, e condividere risorse. Questo attraverso a protocolli a basso livello (TCP/UDP, IP) o alto livello (NFS, SMB).

2.2.1 Struttura del SO

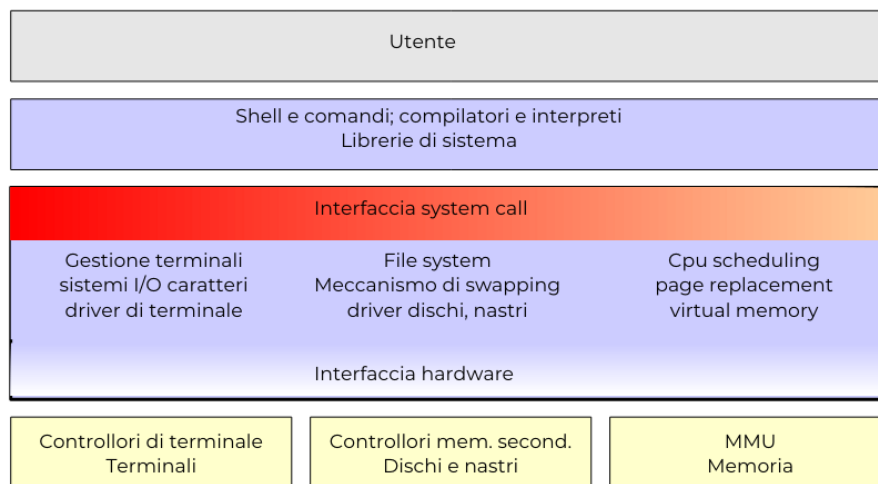
Un SO deve tener conto di efficienza, manutenibilità, espansibilità e modularità.

- Struttura semplice (o senza struttura)
Sono SO semplice che pian piano sono stati fatti update oltre lo scopo originale. Più o come una collezione di procedure che si chiamano a vicenda.



MS-DOS: fortemente legato al hw, le interfacce e i livelli di funzionalità non sono separati, quindi qualsiasi programma poteva fare I/O. Un crash del programma porta il crash del SO.

UNIX: poco più strutturato del precedente ma sempre legato al hw, suddiviso in kernel e programmi di sistema.



- Struttura a strati

Possiamo teorizzare un SO come un insieme di strati, questo ci dà ampia modularità. Avere troppi strati però aggiunge molto overhead. Nella realtà si cerca di avere un numero limitato di strati.

2.2.2 Software engineering del SO

Filosofia che aiuta a progettare sistemi flessibili e manutenibili.

- Politica: parte del sistema che prende le decisioni
- Meccanismo: parte che realizza materialmente le azioni decise dalla politica

Separandoli possiamo modificare una senza toccare l'altra e viceversa.

Microkernel Si implementano nel kernel i soli meccanismi, delegando la gestione della politica a processi fuori dal kernel.

Cosa succede se li si unisce? Un semplice bug grafico può portare il crash di tutto il sistema.

Come definiamo un kernel?

- Kernel monolitici: unico insieme di procedure di gestione mutuamente coordinate e astrazioni dell'HW
Attraverso le System calls implementiamo i servizi forniti dal kernel, che verranno poi eseguiti in kernel mode in moduli. (Linux, FreeBSD UNIX)
- Micro kernel: semplici astrazioni dell'HW gestite e coordinate da un kernel minimale (tipo client/server, message passing)
Rimuovendo dal kernel tutte le parti non essenziali e implementarle come processi a livello utente (per accedere ad un file, un processo interagisce con il processo gestore del file system). Alla base ci serve un meccanismo di comunicazione *message passing* che gestirà il microkernel. Tramite questo possiamo creare la maggior parte delle API dei SO moderni:

```
int open(char* file , ...)
{
    msg = < OPEN, file , ... >;
    send(msg, file-server);
}
```

```

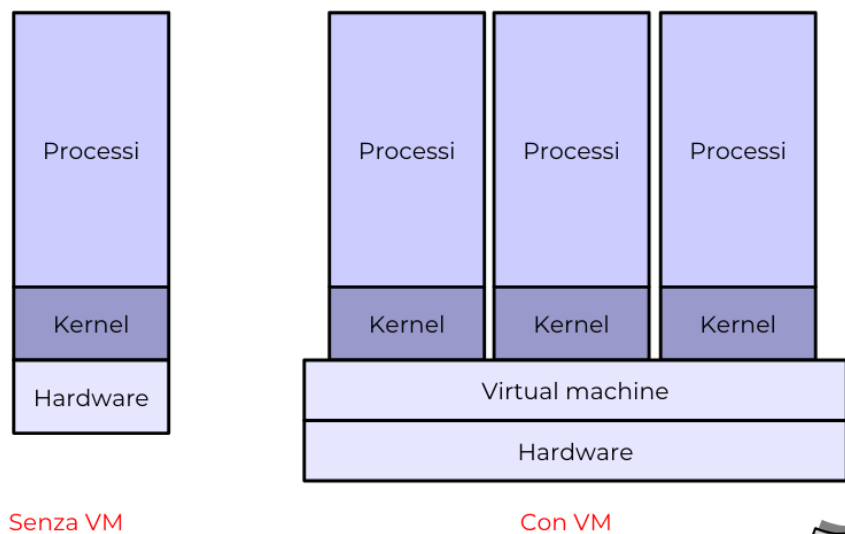
fd = receive(file-server);
return fd;
}

```

- Kernel ibridi: simili ai precedenti ma con più componenti eseguite in kernel space per efficienza.
Sono microkernel che eseguono parte in kernel space, e message passing tra i moduli in user space (famiglia Windows NT)

2.2.3 Macchine virtuali

Emuliamo il funzionamento del hw, possiamo eseguire qualsiasi SO.



I vantaggi sono molteplici (emulazione architetture hw diverse, coesistenza di SO, sistemi mono-task in multitask, ecc) ma a costo di una grande inefficienza e con alta difficoltà di condivisione delle risorse.

2.2.4 Namespace

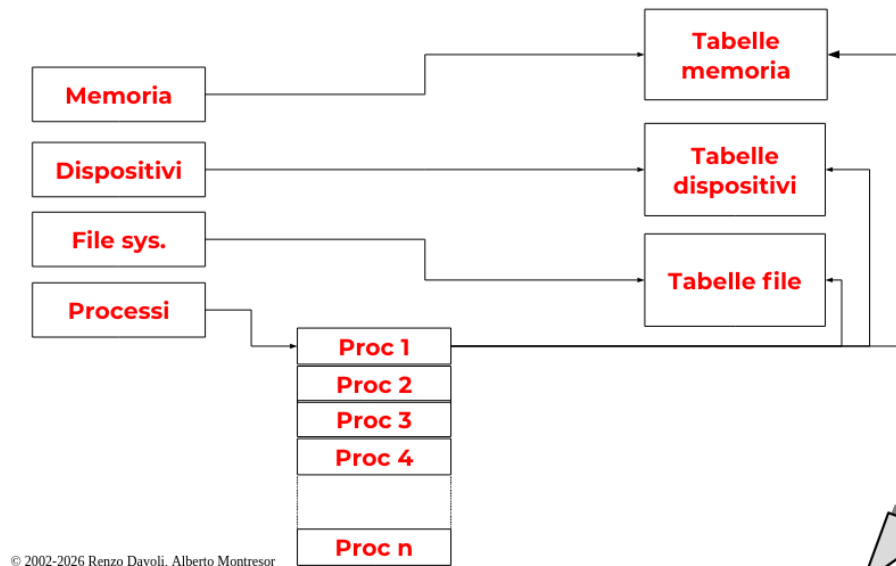
I namespace isolano le risorse in modo che i processi posano vedere solo quelle collegate a essi. (PID, NET, USER, ecc) Li troviamo in Linux, questi sono la base per i container ovvero un gruppo di processi che ha: immagine di file system privata, propria configurazione di rete, limiti di accesso alle risorse; tutto questo gestito da meccanismi di amministrazione a run-time (docker, containerd, ecc).

2.3 Scheduling

Un sistema operativo ha bisogno di strutture dati per mantenere informazioni sulle risorse gestite (processo, mem, principale/secondaria, dispositivi).

Queste strutture dati comprendono:

- tabelle di memoria: allocazione memoria per il SO, sia principale che secondaria
- tabelle di I/O
- tabelle di filesystem: elenco dei dispositivi utilizzati per mantenere il file system, file aperti, ecc
- **tabelle dei processi**



Come è composto un processo per il SO?

- il codice da eseguire
- i dati su cui operare
- uno stack di lavoro per la gestione di chiamate di funzione, passaggio di parametri e variabili locali
- un insieme di attributi contenenti tutte le informazioni necessarie per la gestione del processo stesso

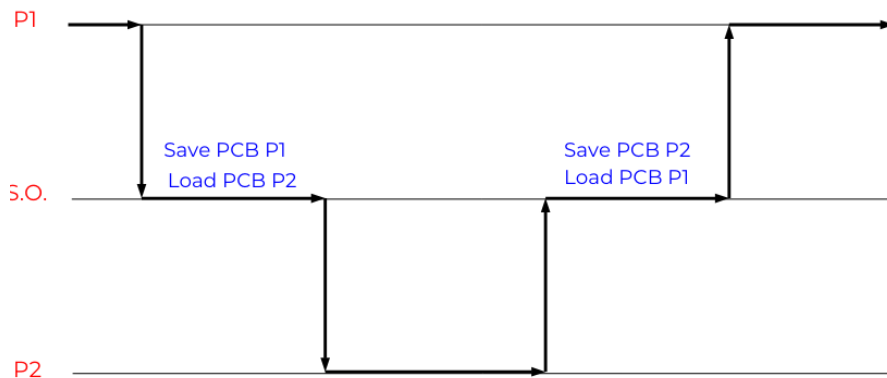
Tutto questo si chiama **process control block** (PCB). Per gestire tutti i processi abbiamo una tabella di PCB (ogni processo ha 1 PCB).

Ogni PCB può essere suddiviso in

- informazioni di identificazione di processo (pid, id utente, altri identificatori)
- informazioni di stato del processo (registri generali e speciali)
- informazioni di controllo del processo
 - informazioni di scheduling come esecuzione, attesa, ecc
 - informazioni per algoritmo di scheduling utilizzato (es. priorità)
 - informazioni gestione memoria
 - informazioni relative alle risorse (file aperti, device, ecc)
 - informazioni per interprocess communication (IPC) come semafori, stato segnali, ecc

2.3.1 Scheduler, processi e thread

Lo **scheduler** è la componente più importante del SO, decide quale processo deve essere in esecuzione, interviene per richieste di I/O. Per questi ultimo come abbiamo visto ogni volta che abbiamo un interrupt avviene un *mode switching*. Se lo scheduler decide di eseguire un altro processo, il sistema è soggetto ad un **context switching**. Quindi lo stato del processo viene salvato nel PCB corrispondente.



Vita di un processo

- Running
- Waiting (aspetta un qualche evento esterno)
- Ready

Nella maggior parte dei SO i processi sono in forma gerardica padre-figli. Quello che non è specificato dal padre viene eridatato dal figlio. Ovviamente un processo può svolgere 1 sola sequenza di istruzione alla volta.

Nei SO però abbiamo anche i processi multithreaded ovvero sequenze di istruzioni parallele. I thread condividono dati, I/O, codice.

E' conveniente? Sì, infatti creare un nuovo thread è molto più veloce che creare un nuovo processo.

Ma come li implementiamo?

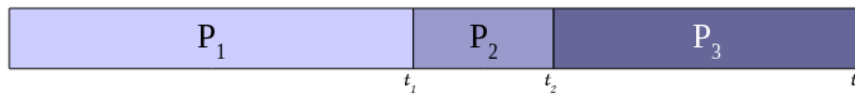
- User thread, implementati attraverso una *thread library* che ci permett di non avere senza alcun intervento del kernel
- Kernel thread, supportati nativamente dal kernel. Più lento perchè si passa da livello utente a livello kernel

I SO moderni li implementano tutti e due, infatti abbiamo tre differenti modelli di multithreading:

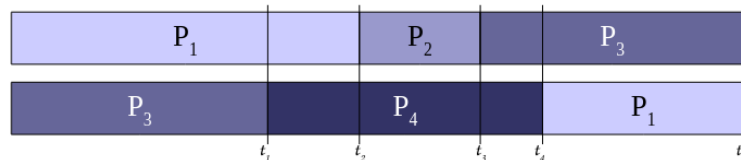
- Many-to-One: un certo numero di user thread vengono mappati su un solo kernel thread
- One-to-One: ogni user thread viene mappato su un kernel thread
- Many-to-Many

2.3.2 Scheduling

Per rappresentare uno schedule si usano i diagrammi di Gantt



In questo esempio da $0 - t_1$ abbiamo la CPU che esegue P_1 , e così via.
Nel caso di Gantt multi-risorsa lo rappresentiamo



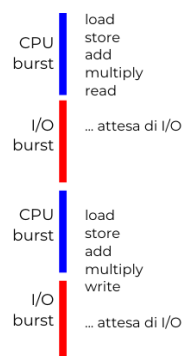
Un **context switch** può avvenire a causa di

- 1. Un processo passa da running a waiting
- 2. Un processo passa da running a ready
- 3. Un processo passa da waiting a ready
- 4. Un processo termina

Nota: per 2 e 3 è possibile continuare ad eseguire il processo corrente.

Se avvengono i context switch solo per 1 e 4 allora abbiamo uno scheduler **non-preemptive** (cooperativo) ovvero se il controllo della risorsa viene trasferito solo se l'assegnatario attuale lo cede volontariamente. Mentre se avviene sempre allora abbiamo uno scheduler **preemptive** ovvero è possibile che il controllo della risorsa venga tolto all'assegnatario attuale a causa di un evento (tutti gli scheduler moderni).

Durante l'esecuzione di un processo abbiamo i momenti di attività della CPU (**CPU burst**) e i momenti di attività I/O (**I/O burst**).



Lo scheduler che algoritmo si basa?

- First Come, First Served: il processo che arriva per primo, viene servito per primo (no-preemption)

- Shortest-Job First: la CPU viene assegnata al processo ready che ha la minima durata del CPU burst successivo (no-preemption)
Sarebbe la soluzione ottimale ma è impossibile applicarla nella realtà! Possiamo solo approssimarla con due algoritmi
 - Shortest-Next-CPU-Burst First (no-preemptive): il processo corrente esegue fino al completamento del suo CPU burst
 - Shortest-Remaining-Time-First (preemptive): il processo corrente può essere messo nella coda ready, se arriva un processo con un CPU burst più breve di quanto rimane da eseguire al processo corrente
- Round-Robin: un processo non può rimanere in esecuzione per un tempo superiore alla durata del **quanto di tempo** (se breve il SO diventa inefficiente, se lungo abbiamo tanti periodi di inattività). E quindi abbiamo due possibilità:
 - un processo può lasciare il processore volontariamente, in seguito ad un'operazione di I/O
 - un processo può esaurire il suo quanto di tempo senza completare il suo CPU burst, nel qual caso viene aggiunto in fondo alla coda dei processi pronti

in entrambi i casi, il prossimo processo da eseguire è il primo della coda dei processi pronti.

Ovviamente queste implementazioni non vanno sempre bene (es. frame video). Allora implementiamo lo **scheduling a priorità**.

Ad ogni processo associamo una specifica priorità, lo scheduler sceglie il processo pronto con priorità più alta. Le priorità sono definite dal SO o dal utente. La priorità può essere statica o dinamica durante la vita di un processo.

Possiamo ulteriormente migliorare attraverso lo **scheduling a classi di priorità** ovvero creare diverse classi di processi con caratteristiche simili e assegnare ad ogni classe specifiche priorità. La coda ready viene quindi scomposta in molteplici "sottocode", una per ogni classe di processi.

Per ogni classe poi possiamo pure utilizzare una politica specifica adatta alle caratteristiche della classe. Così otteniamo lo **scheduling Multilivello**.

Possiamo anche definire lo **scheduling real-time**, ovvero la correttezza dell'esecuzione non dipende solamente dal valore del risultato, ma anche dall'istante temporale nel quale il risultato viene emesso.

2.4 Gestione risorse e deadlock

Le risorse le possiamo suddividere in classi. Le risorse di una classe vengono dette **istanze** della classe. Il numero di risorse in una classe viene detto **molteplicità** del tipo di risorsa.

Un processo non può richiedere una specifica risorsa ma solo una risorsa di una specifica classe. Come le assegniamo?

- Risorse ad assegnazione statica: avviene al momento della creazione del processo e rimane valida fino alla terminazione
- Risorse ad assegnazione dinamica: i processi richiedono, le utilizzano e le rilasciano quando non più necessarie. (I/O)

Tipi di richieste

- Richiesta bloccante: il processo richiedente si sospende se non ottiene immediatamente l'assegnazione
- Richiesta non bloccante: la mancata assegnazione viene notificata al processo richiedente, senza provocare la sospensione

Tipi di risorse

- Risorse non condivisibili: una singola risorsa non può essere assegnata a più processi contemporaneamente (CPU, C.S., stampanti)
- Risorse condivisibili: (file aperti)

Risorse prerilasciabili Una risorsa si dice prerilasciabile (*preemptable*) se la funzione di gestione può sottrarla ad un processo prima che questo l'abbia effettivamente rilasciata. Il processo che subisce il prelascio si deve sospendere, la risorsa prerilasciata sarà successivamente restituita al processo.

La risorsa è prelasciabile se il suo stato non si modifica durante l'utilizzo (oppure il suo stato può essere facilmente salvato e ripristinato).

Risorse non prelasciabili Una risorsa si dice non prelasciabile se la funzione di gestione non può sottrarle al processo al quale sono assegnate. Sono non prerilasciabili le risorse il cui stato non può essere salvato e ripristinato.

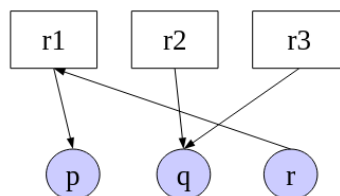
2.4.1 Deadlock

Quando avviene un deadlock? Quando tutte queste condizioni devono valere tutte contemporaneamente affinché un deadlock si presenti nel sistema

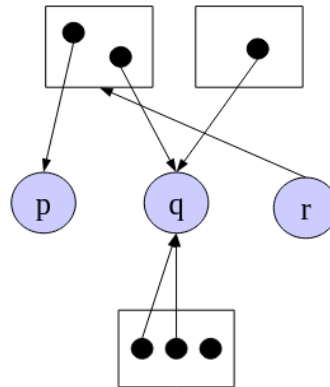
- Mutua esclusione: risorse non condivisibili
- Assenza di prerilascio: le risorse devono essere rilasciate volontariamente dai processi che le controllano
- Richieste bloccanti: un processo che ha già ottenuto risorse può chiederne ancora
- Attesa circolare: il processo P_n aspetta la risorsa controllata da P_{n+1} fino a P_k che aspetta P_0 .

Grafo di Holt Grafo diretto (direzione), bipartito (i nodi sono suddivisi in due sottoinsiemi $\langle \text{risorse}, \text{processi} \rangle$ e non esistono archi che collegano nodi dello stesso sottoinsieme).

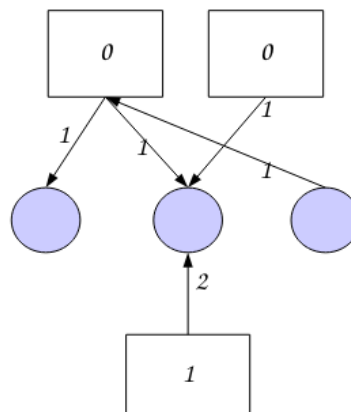
risorsa $\rightarrow$ processo := la risorsa è assegnata al processo
processo $\rightarrow$ risorsa := il processo ha richiesto la risorsa



- Grafo di Holt in generale: l'insieme delle risorse è partizionato in classi e gli archi di richiesta sono diretti alla classe e non alla singola risorsa
 - i processi sono rappresentati da *cerchi*
 - le classi sono rappresentati come *contenitori rettangolari*
 - le risorse come *punti* all'interno delle classi



- Grafo di Holt - Notazione operativa: grafo pesato (con pesi ai nodi risorsa e pesi sugli archi)
 - Sugli archi la molteplicità di risorse o richiesta
 - All'interno delle classi: numero di risorse non ancora assegnate

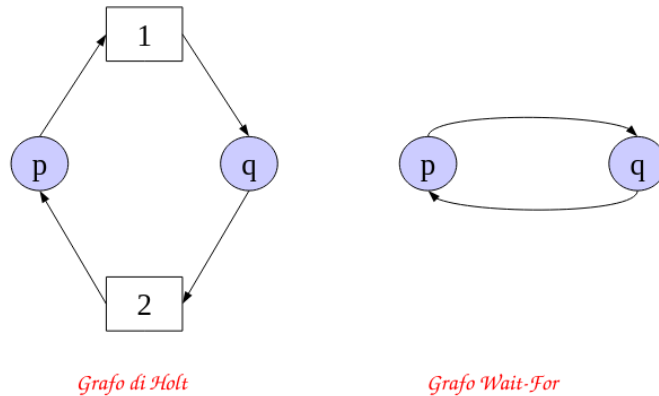


Metodi di gestione dei deadlock

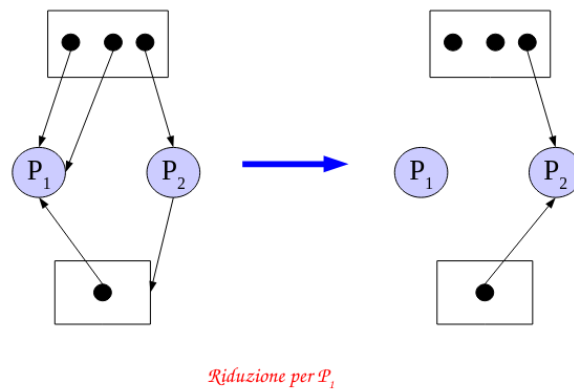
- Deadlock detection and recovery: permettere al SO di entrare in deadlock, poi con un algoritmo lo rileviamo ed eventualmente eseguire un'azione di recovery. Mantenere aggiornato il grafo di Holt, registrando su di esso tutte le assegnazioni e le richieste di risorse. Utilizzare il grafo per rilevare gli stati di deadlock.

- Sistema centralizzato: **Deadlock detection con grafo di Holt**

Teorema: se le risorse sono a richiesta bloccante, non condivisibili e non prerilasciabili, lo stato è di deadlock se e solo se il grafo di Holt contiene un ciclo



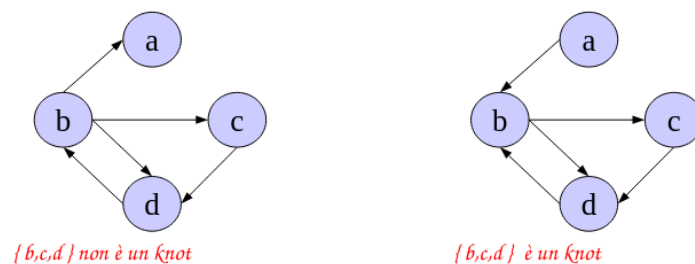
Un grafo di Holt si può semplificare omettendo i nodi processi con solo archi entranti.



Teorema: se le risorse sono a richiesta bloccante, non condivisibili e non prerilasciabili, non abbiamo deadlock se e solo se il grafo di Holt è completamente riducibile.

- Sistema distribuito: **Deadlock detection - Knot**

Teorema: Dato un grafo di Holt con una sola richiesta sospesa per processo, se le risorse sono a richiesta bloccante, non condivisibili e non prerilasciabili, allora il grafo rappresenta uno stato di deadlock se e solo se esiste un knot.



Knot: Partendo da un qualunque nodo di M ($\neq \emptyset$ t.c. $\forall n \in M R(n) = M$ con $R(n)$ insieme di raggiungibilità di n), si possono raggiungere tutti i nodi di M e nessun nodo all'infuori di esso.

– **Deadlock recovery**

La soluzione può essere:

- * **Manuale:** l'operatore viene informato e eseguirà alcune azioni che permettano al sistema di proseguire
 - * **Automatica:** il sistema operativo è dotato di meccanismi che permettono di risolvere in modo automatico la situazione, in base ad alcune politiche
 - Terminazione totale: tutti i processi coinvolti vengono terminati
 - Terminazione parziale: viene eliminato un processo alla volta, fino a quando il deadlock non scompare
 - Checkpoint/rollback: lo stato dei processi viene periodicamente salvato su disco (*checkpoint*), in caso di deadlock, si ripristina (*rollback*) uno o più processi ad uno stato precedente, fino a quando il deadlock non scompare
- Deadlock prevention / avoidance: impedire al sistema di entrare in uno stato di deadlock

– **Prevention:** Per evitare il deadlock si elimina una delle quattro condizioni del deadlock. Vediamo quale attaccare:

- * **Mutua esclusione:** non è possibile, alcune risorse sono intrinsecamente non condivisibili
 - * **Richiesta bloccante:** l'*allocazione totale* non è sempre possibile
 - * **Assenza di prerilascio:** non è possibile, alcune risorse sono intrinsecamente non prelaschiabili. Servono interventi manuali
 - * **Attesa Circolare:** attraverso una allocazione *gerardica* alle classi di risorse vengono associati valori di priorità, e i processi possono richiedere risorse solo in ordine crescente di priorità
- **Avoidance:** prima di assegnare una risorsa ad un processo, si controlla se l'operazione può portare al pericolo di deadlock, se positivo l'operazione viene ritardata.

Algoritmo del banchiere: il banchiere deve essere in ogni istante in grado di soddisfare tutte le richieste dei clienti, o concedendo immediatamente il prestito oppure comunque facendo loro aspettare la disponibilità del denaro in un tempo finito.

Definiamo uno stato **SAFE** se esiste una sequenza di processi $P_1, P_2, \dots, P_n$ tale che per ogni P_i le risorse richieste da P_i sono soddisfatte dalle risorse attualmente disponibili più le risorse detenute dai processi $P_1, P_2, \dots, P_{i-1}$. Definiamo uno stato **UNSAFE** se non è SAFE.

Lo stato SAFE può essere verificato usando la sequenza che ordina in modo crescente i valori di n_i (risorse ancora necessarie a P_i per completare) e verificando che per ogni processo P_i della sequenza, $n_{S(j)} \leq \text{avail}[j]$, con $j = 1, \dots, N$ (risorse attualmente disponibili).

Algoritmo del banchiere - Esempio stato SAFE

Capitale Iniziale	IC	150
Saldo di cassa	COH	0

la sequenza 3,2,1,4,5 consente il soddisfacimento di tutte le richieste

Cliente	MAX	Prestito Attuale	Credito residuo
i	c_i	p_i	n_i
1	100	70	30
2	20	10	10
3	30	30	0
4	50	10	40
5	70	30	40

Teorema dell'algoritmo del banchiere Siano C, C' le sequenze di processi. Sia $H = \{p \in \text{processi} \mid p \notin C\}$. Sia h il primo elemento di H che compare in C' . Tutti gli elementi di C' prima di h sono in C . Chiamiamo C'' il segmento iniziale di C' fino al punto precedente l'applicazione di h . Le risorse disponibili all'applicazione di h in C' sono $avail(C'') = COH + \sum_{j \in C} p_j$; h e' applicabile quindi $n \leq avail(C'')$. Ma $avail(C) = COH + \sum_{j \in C} p_j$ quindi se $avail(C) \leq avail(C'')$ allora h e' applicabile alla fine di C contro l'ipotesi che C non sia ulteriormente estendibile (SAFE).

- Ostrich algorithm (Algoritmo dello struzzo): ignorare il problema del tutto! Calcolare il costo dei deadlock puo' essere troppo elevato. (Unix, JVM)

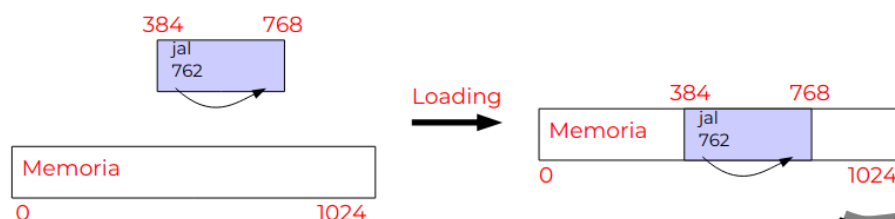
2.5 Gestione della memoria

La parte del sistema operativo che gestisce la memoria principale si chiama **memory manager**. Deve tenere traccia della memoria libera e occupata, allocare memoria ai processi e deallocarla quando non serve piu'.

2.5.1 Binding, loading, linking

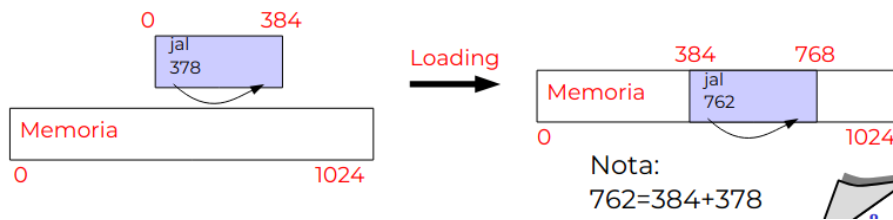
Con il termine binding si indica l'associazione di indirizzi logici di memoria (e.g. nomi di variabili, label) ai corrispondenti indirizzi fisici. Questa avviene a:

- **Tempo di compilazione:** gli indirizzi vengono calcolati al momento della compilazione e resteranno gli stessi ad ogni esecuzione del programma. Non e' compatibile con la multiprogrammazione.



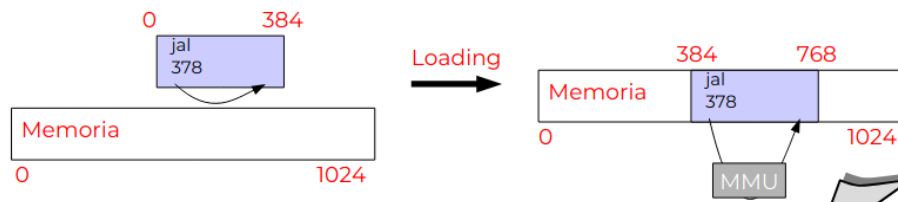
Questo e' leggero, semplice e non richiede hw speciale.

- **Tempo di caricamento:** il codice generato dal compilatore non contiene indirizzi assoluti ma relativi



Funziona in multiprogrammazione, non serve hw speciale ma richiede una traduzione degli indirizzi da parte del loader, e quindi formati particolare dei file eseguibili.

- **Tempo di esecuzione:** l'individuazione dell'indirizzo di memoria effettivo viene effettuata durante l'esecuzione da un componente hardware apposito: la memory management unit (MMU)



Prestazioni speciali

- **Loading dinamico:** caricare alcune routine di libreria solo quando vengono richiamate (ddl). Tutte le routine a caricamento dinamico risiedono su un disco (codice rilocabile), quando servono vengono caricate, dipende dal programmatore quali e come utilizzarle.
- **Linking dinamico:** E' possibile posticipare il linking delle routine di libreria al momento del primo riferimento durante l'esecuzione, ovvero collegare le librerie in fase di esecuzione invece che durante la compilazione. Consente di avere eseguibili più compatti.

2.5.2 Allocazione contigua

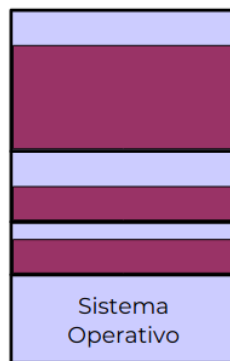
- **Allocazione contigua:** tutto lo spazio assegnato ad un processo deve essere formato da celle consecutive.
- **Allocazione non contigua:** è possibile assegnare a un processo aree di memorie separate
- **Allocazione statica:** un processo deve mantenere la propria area di memoria dal caricamento alla terminazione
- **Allocazione dinamica:** durante l'esecuzione, un processo può essere spostato all'interno della memoria

Ora vediamo diverse tipologie di allocazione:

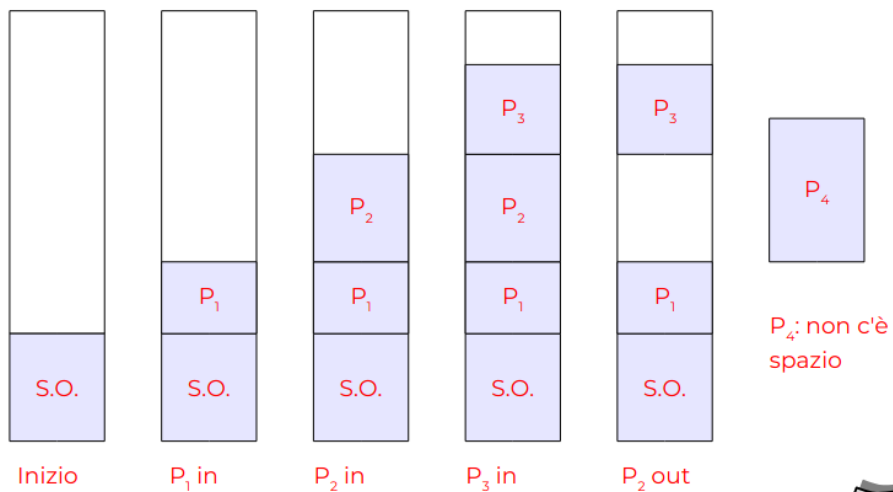
- **Allocazione a partizioni fisse (contigua-statica):** è possibile utilizzare una coda per partizione, oppure una coda comune per tutte le partizioni. Va bene per SO mono-programmati (MS-DOS, sistemi embedded)



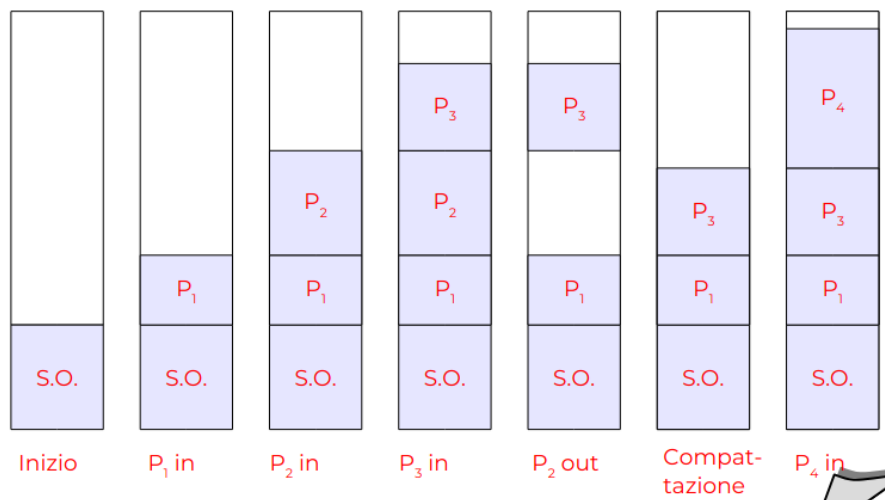
- **Frammentazione interna:** se un processo occupa una dimensione inferiore a quella della partizione che lo contiene, lo spazio non utilizzato è sprecato. La presenza di spazio inutilizzato all'interno di un'unità di allocazione si chiama **frammentazione interna**.



- **Allocazione a partizioni dinamiche (contigua-statica):** la memoria disponibile (nella quantità richiesta) viene assegnata ai processi che ne fanno richiesta

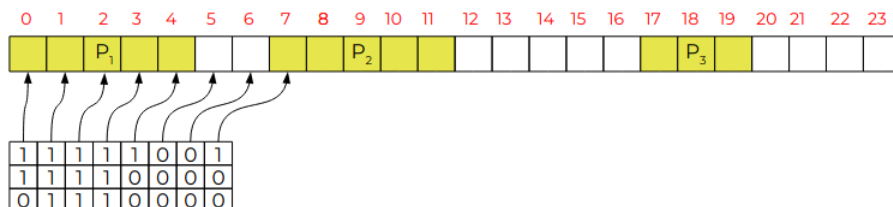


- **Frammentazione esterna:** dopo un certo numero di allocazioni e deallocazioni di memoria dovute all'attivazione e alla terminazione dei processi lo spazio libero appare suddiviso in piccole aree, il fenomeno è detto **frammentazione esterna**.
- **Compattazione:** se è possibile relocare i processi durante la loro esecuzione, è allora possibile procedere alla **compattazione** della memoria. Risolve il problema della frammentazione esterna.



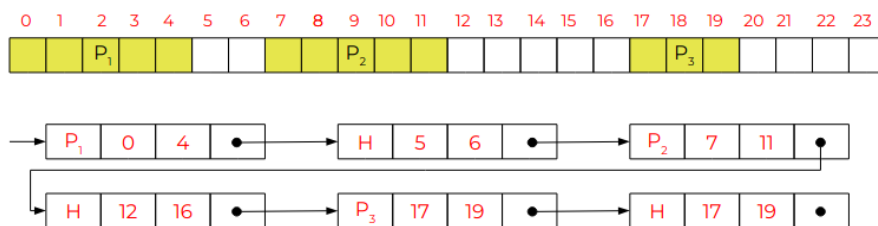
- Strutture di dati (dinamica):

- Mappa di bit: memoria suddivisa in blocchi, per ogni blocco un bit per indicare se occupata (1) o no (0).



Il costo per trovare un insieme di blocchi contigui liberi e' $O(n)$.

- Lista con puntatori: una lista che segna per ogni L se e' occupata P o libera H , viene segnata anche la dimensione del blocco $\langle inizio, fine \rangle$.



Per allocare un processo di dimensione n si scorre la lista fino a trovare un blocco libero di dimensione $m \geq n$. Poi la vecchia lista H diventa $P \rightarrow H'$, con H' di dimensione $m - n$ blocchi. Costo: $O(n)$

Per deallocare, se abbiamo altri H vicini si crea un unico blocco libero accorpandoli. Costo: $O(1)$.

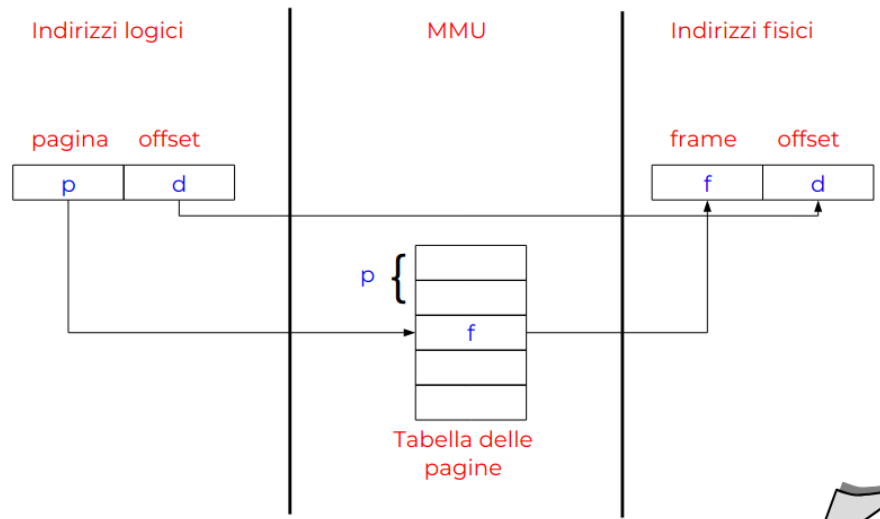
Nella allocazione dinamica come scegliere il blocco libero?

- **First fit:** si scorre la lista dei blocchi liberi e si assegna il primo blocco di dimensione $m \geq n$ trovato
- **Next fit:** come First Fit, ma invece di ripartire sempre dall'inizio, parte dal punto dove si era fermato all'ultima allocazione. Evita frammentazione all'inizio della memoria, ma non è molto efficiente se la memoria è molto frammentata. Ha performance peggiori rispetto a First Fit.

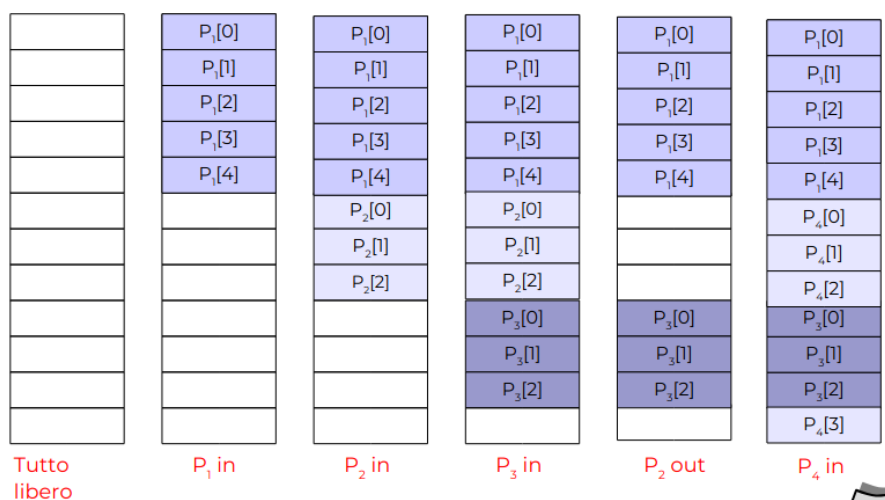
- **Best fit:** si scorre tutta la lista dei blocchi liberi e si assegna il blocco di dimensione $m \geq n$ più piccolo possibile. Riduce la frammentazione esterna, ma è più lento di First Fit.
- **Worst fit:** elezione il più grande fra i blocchi liberi presenti in memoria. Riduce la frammentazione esterna, ma è più lento di First Fit.

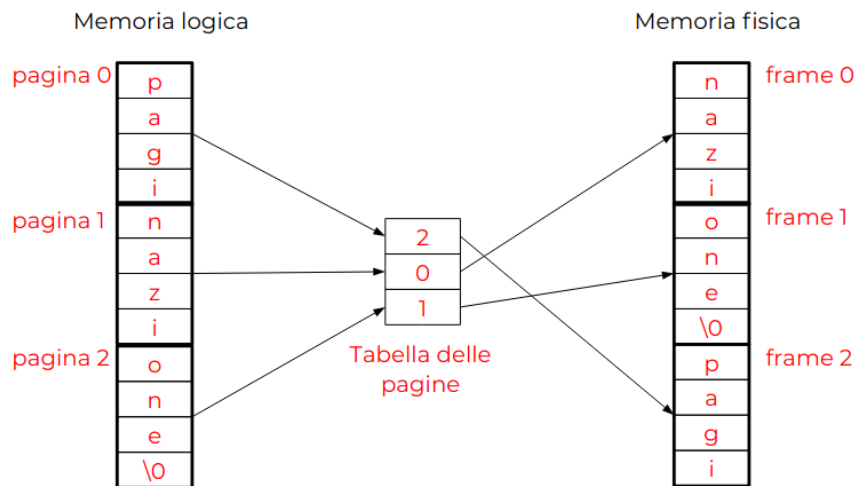
2.5.3 Paginazione

Approccio moderno per ridurre la frammentazione interna ed elimina quella esterna. Però serve hw speciale (MMU).



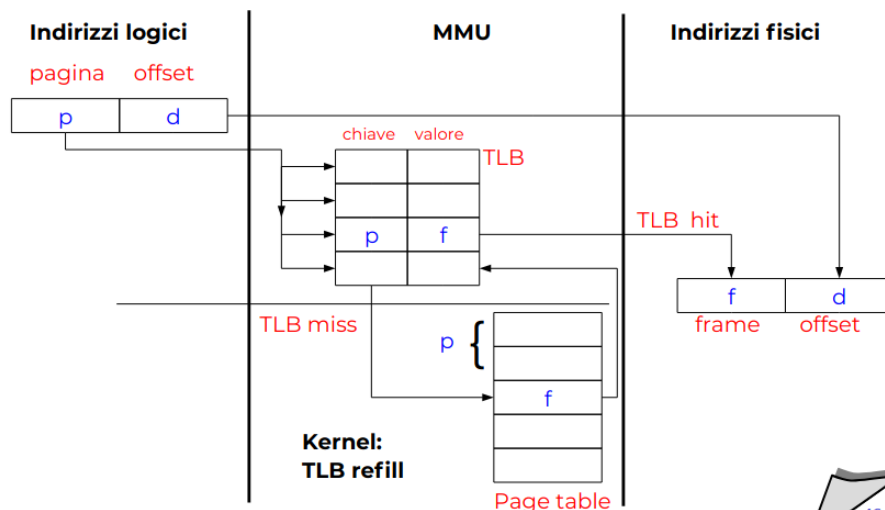
- La memoria fisica e' divisa in **frame**, ovvero blocchi di dimensione fissa. Deve essere una potenza di 2, tipicamente 4KB.
Pagine troppo piccole portano a una tabella grande, mentre pagine troppo grandi portano a frammentazione interna.
- La memoria logica e' divisa in **pagine** della stessa dimensione dei frame.
- Quando un processo chiede memoria, si allocano un numero sufficienti di frame (anche non contigui).





Dove metiamo la tabella delle pagine?

- Registri dedicati: un insieme di registri all'interno del MMU. Questo però è troppo costoso.
- Memoria: il numero di accessi in memoria però raddoppia, prima alla tabella e poi al dato.
- Translation lookaside buffer (**TLB**): insieme di registri (solo 8-2048 registri) suddivisi in due parti $\langle \text{chiave}, \text{valore} \rangle$. Per cercare un dato cerchiamo la sua chiave e se presente (*TLB hit*) si ritorna il valore; in caso contrario (*TLB miss*) si accede alla tabella in memoria e si ritorna il valore.

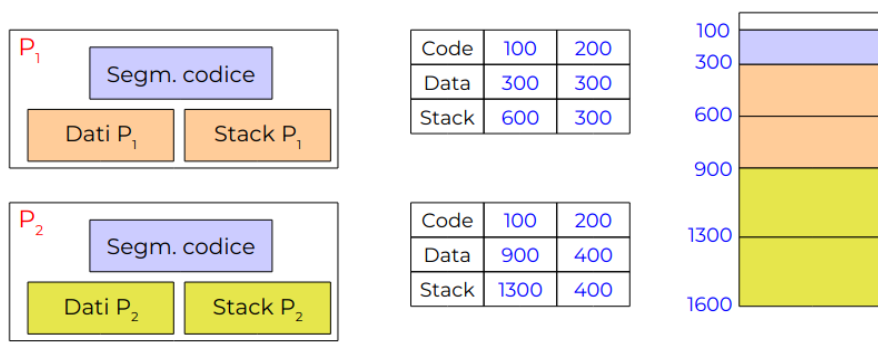


Ovviamente questo modello di cache come tutti si basa sul principio di località'.

2.5.4 Segmentazione

In un sistema a segmentazione la memoria di un processo è suddivisa in aree differenti. Quindi lo spazio di indirizzamento logico è dato da un insieme di segmenti (aree di memoria continua con $\langle \text{nome segmento}, \text{lunghezza} \rangle$). Questo però spetta al programmatore/compilatore la suddivisione dei programmi in segmenti.

Segmentazione e condivisione? E' possibile condividere un segmento tra più processi, ad esempio il segmento di codice di una libreria condivisa o la comunicazione fra i processi.



Il problema della segmentazione e' che se abbiamo segmenti di **dimensione variabile** allora soffriamo di frammentazione esterna, mentre se abbiamo segmenti di **dimensione fissa** allora soffriamo di frammentazione interna. Potremmo teoricamente utilizzare tecniche di allocazione dinamica come First Fit. Ma abbiamo gli stessi problemi di frammentazione della allocazione dinamica.

Segmentazione + paginazione E se combinassimo le due tecniche? Ogni segmento viene suddiviso in pagine che vengono allocate in frame liberi della memoria (non necessariamente contigui).

A livello hw dobbiamo avere una MMU che supporti sia la segmentazione che la paginazione. Così avremo però sia i benefici della segmentazione (protezione, condivisione) che quelli della paginazione (no frammentazione esterna).

2.5.5 Memoria virtuale

E' la tecnica che permette l'esecuzione di processi che non sono completamente in memoria. Questo permette di eseguire processi che richiedono più memoria di quella fisica disponibile. A contro però se implementata male, la memoria virtuale, può rallentare un sistema.

Non infrange i requisiti di un'architettura di Von Neumann? No, perché l'architettura di Von Neumann dice che le istruzioni da eseguire e i dati su cui operano devono essere in memoria, ma non e' detto che l'intero spazio di indirizzamento logico sia in memoria o sia utilizzato del tutto.

Implementazione della memoria virtuale Ogni processo ha uno spazio di *indirizzamento virtuale* (anche più grande di quello fisico se serve). Gli indirizzi logici virtuali possono essere mappati su indirizzi fisici della memoria principale oppure su memoria secondaria.

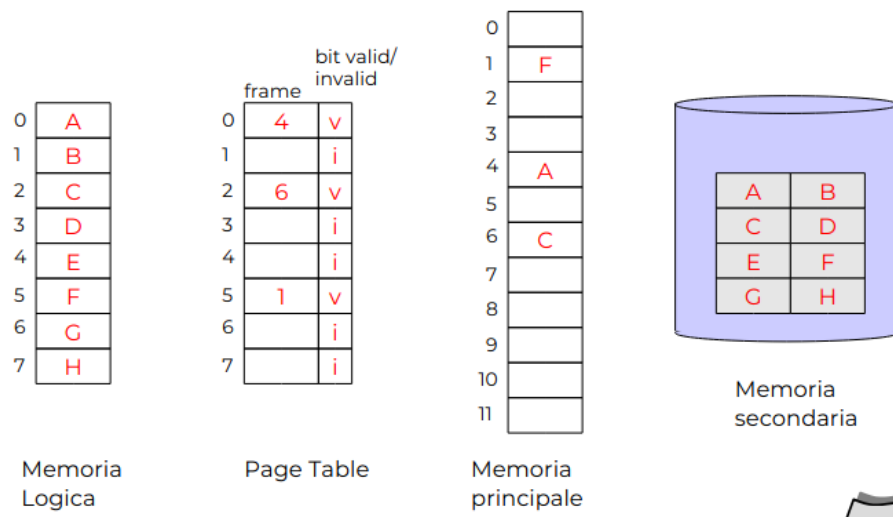
In caso di accesso a dati su memoria secondaria i dati vengono trasferiti su memoria principale o se fosse piena si sposta in memoria secondaria i dati contenuti in memoria principale che sono considerati meno utili (swap).

Demand paging

La **paginazione a richiesta** utilizza la tecnica di paginazione ammettendo però che alcune pagine possano essere in memoria secondaria.

Ci segniamo nelle tabella delle pagina se una pagina e' in memoria principale o secondaria, attraverso un bit v (*valid bit*).

Cosa succede se accediamo a una pagina che non e' in memoria? Il processore genera un trap **page fault** e il **pager** (componente del SO) si occupa di caricare la pagian mancante e di aggiornare la tabella delle pagine.



Pager

Il pager (in passato **swapper**) si occupa di gestire la **swap area** (l'area del disco utilizzata per ospitare le pagine in memoria secondaria).

- In passato avevamo lo **swap** ovvero copiare l'intera area are di memoria di in processo da mem. principale alla secondaria (*swap-in*) o l'inverso (*swap-out*).
- Potremmo anche utilizzare una tecnica di **paginazione su richiesta**, ovvero carichiamo solo quello che serve. Potrebbe funzionare ma e' una tecnica di *lazy swap*.

Gestione dei page fault

In caso di mancanza di frame liberi che facciamo? Dobbiamo liberare il frame "*meno utile*". Possiamo utilizzare un **algoritmo di rimpiazzamento**, ovvero minimizzare il numero di page fault.

Algoritmo del meccanismo di demand paging:

- Individua la pagina di memoria secondaria'
- Individua un frame libero
- Se il frame libero non esiste
 - richiamo algoritmo di rimpiazzamento
 - aggiorna tabella delle pagfine

- se la pagina "vittima" era stata modificata, aggiorniamola sul disco
- aggiorna tabella dei frame segnando che abbiamo il frame libero
- Aggiorna la tabella dei frame segnando che ora quel frame e' occupato
- Leggiamo la pagina richiesta
- Aggiorniamo la tabella delle pagine
- Riattiva il processo

Algoritmo di rimpiazzamento

- Algoritmo FIFO: quando dobbiamo liberare un frame liberiamo il primo frame che e' stato caricato in memoria. Questo non richiede hw specifico, e' veloce ma alcune volte scarta pagine utilizzate.
Provoca un effetto dove +pagine $\Rightarrow$ +page fault
- Algoritmo MIN - Ottimale: algoritmo teorico che seleziona come pagina vittima una pagina che non sarà più acceduta o la pagina che verrà acceduta nel futuro più lontano
- Algoritmo LRU (Least Recently Used): seleziona come pagina vittima la pagina che è stata usata meno recentemente nel passato.
Per implementarla serve hw specifico, la MMU deve registrare nella tabella delle pagine un time-stamp quando accede ad una pagina, il time-stamp può essere implementato come un contatore che viene incrementato ad ogni accesso in memoria.

Puo' essere implementata via uno stack (LIFO) di pagine, nel quale che tutte le volte che una pagina viene acceduta, viene rimossa dallo stack (se presente) e posta in cima allo stack stesso.

Teorema: un algoritmo a stack non genera casi di anomalia di Belady. Se consideri l'insieme delle pagine in memoria per un dato numero di frame, questo insieme è sempre un sottoinsieme dell'insieme delle pagine che avresti con un numero maggiore di frame.

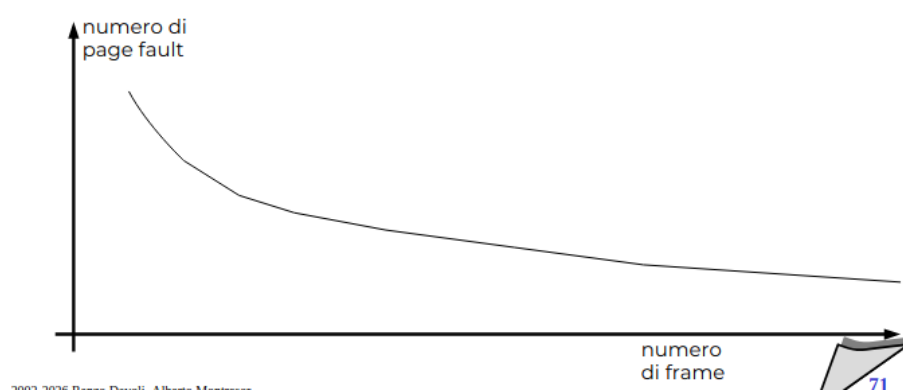
$$S_t(s, A, m) \subseteq S_t(s, A, m + 1)$$

con $S_t(s, A, m)$ insieme della pagine mantenute al tempo t dell'algoritmo A , data una memoria di m frame relativamente alla stringa s .

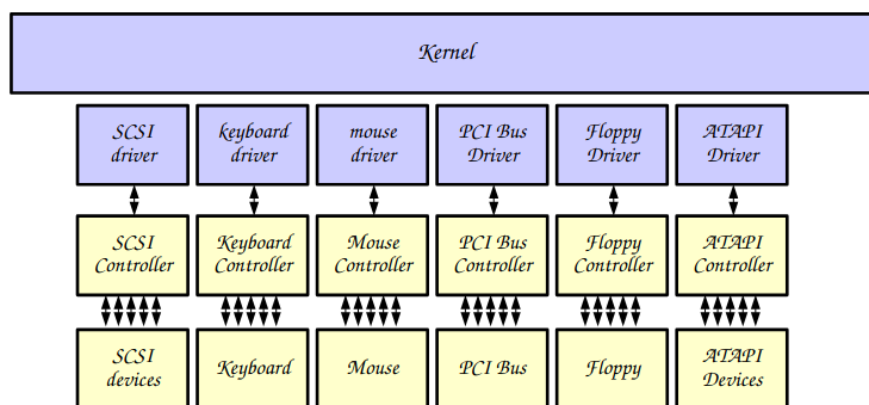
L'algoritmo *LRU* e' a stack. Vediamo come implementarlo:
pagina 87

Definiamo l'**anomalia di Belady**: quando all'aumentare delle pagine aumentano i page fault, come per l'algoritmo FIFO.

Non e' sempre detto che all'aumentare delle pagine aumentino i page fault come in FIFO. Dipende tutto dal algoritmo scelto. Infatti per *Algoritmo MIN - Ottimale* o per *LRU* all'aumentare delle pagine diminuiscono i page fault.



2.6 Gestione I/O, Memoria secondaria



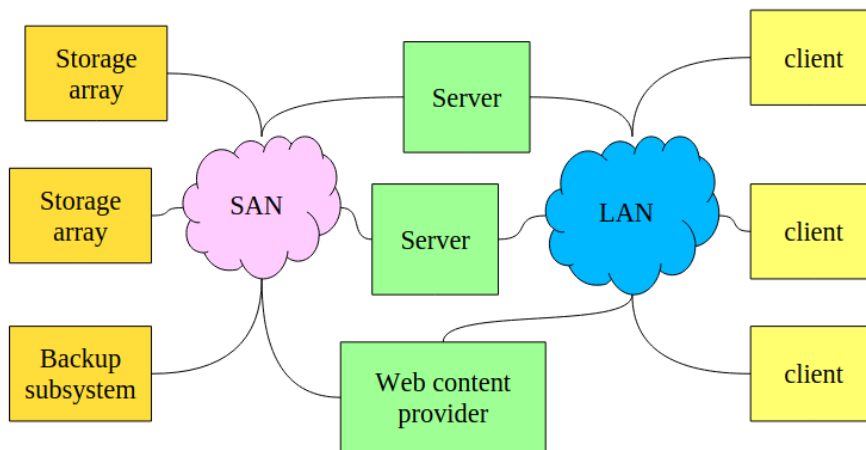
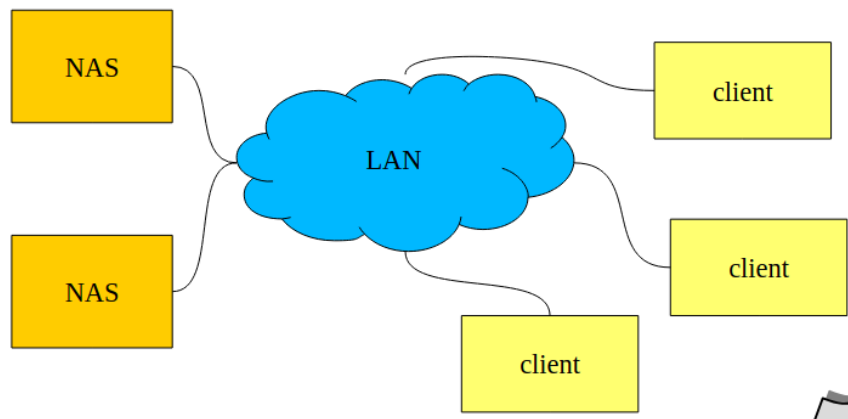
Come classifichiamo un sistema I/O? La modalita' di trasferimento puo' essere a **blocchi** o **caratteri**.

- Blocchi: I dati vengono scritti/letti a blocchi (512byte), tramite filesystem e memory-mapped I/O (contenuto di un file viene mappato in memoria).
- Caratteri: I dati vengono letti/scritti un carattere alla volta, bufferizzazione di una linea alla volta.

2.6.1 Progettazione del sistema di I/O

- Buffering: per gestire una differenza di velocità tra il produttore e il consumatore di un certo flusso di dati, gestire la differenza di dimensioni nell'unità di trasferimento
- Caching: mantiene una copia in memoria primaria di informazioni che si trovano in memoria secondaria (nel buffer troviamo una istanza della informazione, qua una copia!)
- Spooling: buffer che mantiene output per un dispositivo che non può accettare flussi di dati distinti (es. stampanti)
- I/O scheduling

- **Unità che consentono mount remoto**
- **Memoria secondaria condivisa**



2.6.2 Memoria Secondaria

SSD Gli SSD (Solid State Disk) consumano meno energia dei HDD, non hanno fragilità meccanica, hanno un ciclo massimo di scritture. Si legge a blocchi e si legge a **banchi** (molti blocchi), questo porta $V_{\text{lettura}} \gg V_{\text{scrittura}}$ e un accesso uniforme a tutto lo spazio di memoria.

Dischi Un disco è composto da un insieme di piatti, suddivisi in tracce, le quali sono suddivise in settori.

- r velocità di rotazione in *rpm*, 5400/7200/10k rpm
- T_s tempo di seek (tempo medio necessario affinché la testina si sposti sulla traccia desiderata), circa 8-10ms
- V_r velocità di trasferimento, espressa in byte al secondo

Caratteristiche dei dischi:

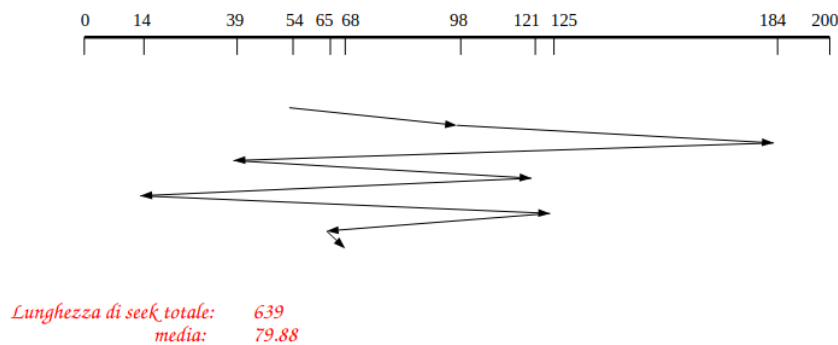
- Il *tempo di accesso* è il tempo necessario per leggere un settore del disco, questo si calcola con tempo di seek + ritardo rotazionale e tempo di trasferimento.
- Il *Ritardo rotazionale* è il tempo medio necessario affinché il settore desiderato arrivi sotto la testina, ovvero $\frac{1}{2r}$

- Il *Transfer time*: dipende dalla quantità dei dati b da leggere, ovvero $\frac{b}{V}$

Nel SO troviamo il **gestione software dei dischi**, ovvero per rendere più efficiente le richieste pendenti utilizziamo un ordine che minimizza le operazioni che richiedono molto tempo (seek). Vediamone alcuni:

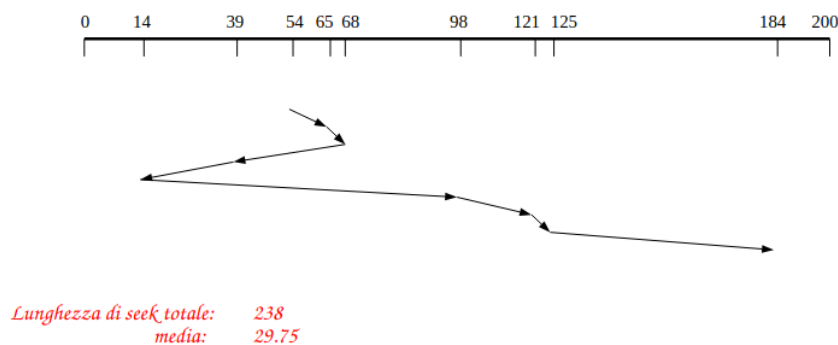
- FCFS (First Come, First Served)/FIFO: non minimizza il numero di seek, non genera starvation

- **Coda delle richieste: 98, 184, 39, 121, 14, 125, 65, 68**
- **Posizione iniziale: 54**



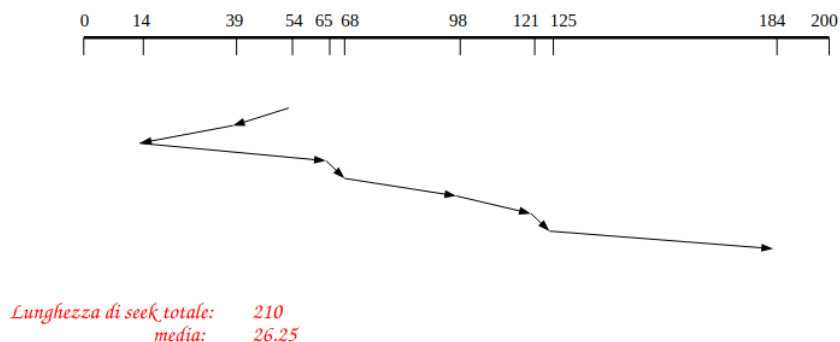
- SSTF (Shortest Seek Time First): seleziona la richiesta che prevede il minor spostamento della testina dalla posizione corrente, può provocare starvation

- **Coda delle richieste: 98, 184, 39, 121, 14, 125, 65, 68**
- **Posizione iniziale: 54**



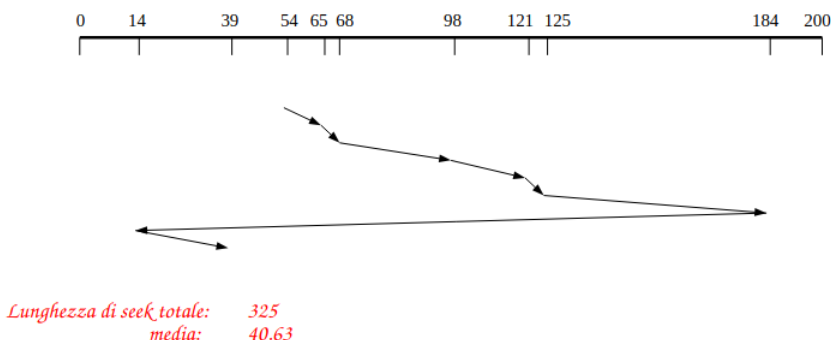
- LOOK (algoritmo dell'ascensore): efficiente, privilegia le tracce centrali, non ha starvation
 - ad ogni istante, la testina è associata ad una direzione
 - la testina si sposta di richiesta in richiesta, seguendo la direzione scelta
 - quando si raggiunge l'ultima richiesta nella direzione scelta, la direzione viene invertita e si eseguono le richieste nella direzione opposta

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54



- C-LOOK: stesso principio di funzionamento del metodo LOOK, ma la scansione del disco avviene in una sola direzione
 - quando si raggiunge l'ultima richiesta in una direzione, la testina si sposta direttamente alla prima richiesta

- **Coda delle richieste:** 98, 184, 39, 121, 14, 125, 65, 68
- **Posizione iniziale:** 54



Nota: sia per LOOK e C-LOOK è possibile che il braccio della testina non si muova per un periodo considerevole di tempo. Per risolvere:

- la coda delle richieste può essere suddivisa in due sottocode separate
- mentre il disk scheduler sta soddisfacendo le richieste di una coda, le richieste che arrivano vengono inserite nell'altra
- quando tutte le richieste della prima coda sono state esaurite, si scambiano le due code

2.6.3 RAID

La velocità dei processori cresce secondo la legge di Moore, la velocità dei dispositivi di memoria secondaria molto più lentamente. Come risolviamo?

Per aumentare la velocità di un componente, una delle possibilità è quella di utilizzare il parallelismo!

Utilizzare un **array di dischi** indipendenti, che possano gestire più richieste di I/O in parallelo.

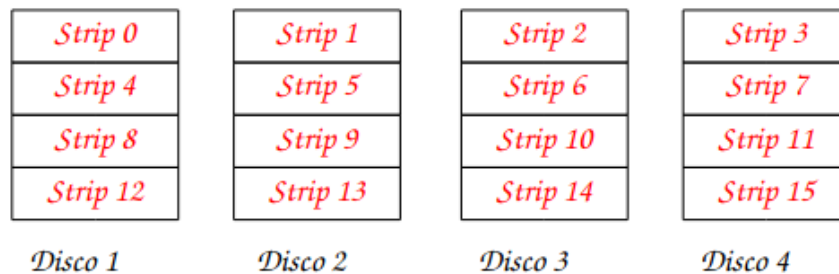
Il RAID (Redundant Array of Independent Disks) consiste in 7 schemi diversi che rappresentano diverse architetture di distribuzione dei dati.

La capacità ridondante dei dischi può essere utilizzata per memorizzare informazioni di parità, che garantiscono il recovery dei dati in caso di guasti.

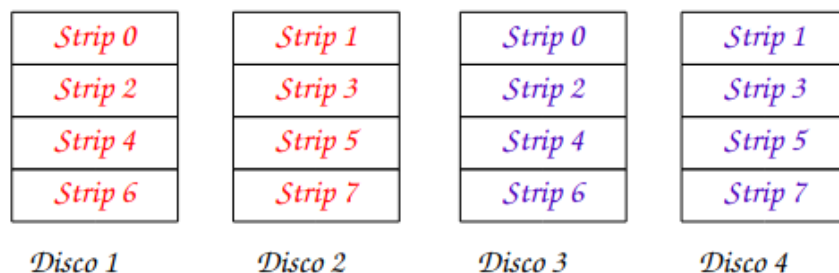
Il SO deve presentare al disco richieste che possano essere soddisfatte in modo efficiente per ottimizzare per performance, tipo richieste di lettura di grandi quantità di dati sequenziali.

RAID 0 (striping) Non possiede meccanismi di ridondanza, i dati vengono distribuiti su più dischi. Se abbiamo richieste su due blocchi indipendenti c'è la possibilità che siano su dischi differenti e quindi le due richieste possono essere servite in parallelo.

Il sistema RAID viene visto come un disco logico, i dati nel disco logico vengono suddivisi in strip (multiplo di settori/blocchi). Strip consecutivi sono distribuiti su dischi diversi, aumentando le performance della lettura dei dati sequenziali.



RAID 1 (mirroring) La ridondanza è ottenuta duplicando tutti i dati su due insiemi indipendenti di dischi, basato su striping ma questa volta uno strip viene scritto su due dischi diversi



Una richiesta di lettura può essere servita da uno qualsiasi dei dischi che ospitano il dato, può essere scelto quello con tempo di seek minore.

Una richiesta di scrittura deve essere servita da tutti i dischi che ospitano il dato, dipende dal disco con tempo di seek maggiore.

RAID 4 Si utilizza il meccanismo di data striping, con strip relativamente grandi. Strip di parità calcolato a partire dagli strip di dati corrispondenti, calcolato bit-per-bit. Lo strip di parità viene posto sul disco di parità.

<i>Strip 0</i>	<i>Strip 1</i>	<i>Strip 2</i>	<i>Strip 3</i>	<i>P(0-3)</i>
<i>Strip 4</i>	<i>Strip 5</i>	<i>Strip 6</i>	<i>Strip 7</i>	<i>P(4-7)</i>
<i>Strip 8</i>	<i>Strip 9</i>	<i>Strip 10</i>	<i>Strip 11</i>	<i>P(8-11)</i>
<i>Strip 12</i>	<i>Strip 13</i>	<i>Strip 14</i>	<i>Strip 15</i>	<i>P(12-16)</i>
<i>Disco 1</i>	<i>Disco 2</i>	<i>Disco 3</i>	<i>Disco 4</i>	<i>Disco 5</i>

RAID 5 Rispetto al RAID4 abbiamo il vantaggio è che non esiste un disco di parità che diventa un bottleneck

<i>Strip 0</i>	<i>Strip 1</i>	<i>Strip 2</i>	<i>Strip 3</i>	<i>P(0-3)</i>
<i>Strip 4</i>	<i>Strip 5</i>	<i>Strip 6</i>	<i>P(4-7)</i>	<i>Strip 7</i>
<i>Strip 8</i>	<i>Strip 9</i>	<i>P(8-11)</i>	<i>Strip 10</i>	<i>Strip 11</i>
<i>Strip 12</i>	<i>P(12-16)</i>	<i>Strip 13</i>	<i>Strip 14</i>	<i>Strip 15</i>
<i>Disco 1</i>	<i>Disco 2</i>	<i>Disco 3</i>	<i>Disco 4</i>	<i>Disco 5</i>

RAID 6 Come RAID 5, ma si utilizzano due strip di parità invece di uno. Questo aumenta l'affidabilità.

2.7

3 Linguaggio C per Sistemi Operativi

Il linguaggio C è un linguaggio di programmazione imperativo, strutturato e compilato, nato negli anni '70 per lo sviluppo di sistemi operativi, in particolare UNIX.

Il Kernel è un componente obbligatorio dei SO moderni, vediamo di cosa gestisce:

- Esegue le funzioni di sistema critiche e interagisce con l'hardware (gestione CPU, memoria per i processi, fs, dispositivi, ecc)
- Caricato in memoria durante il boot e rimane sempre in memoria fisica

Ovviamente questo non basta per definire un SO, perciò dobbiamo avere anche altre utility

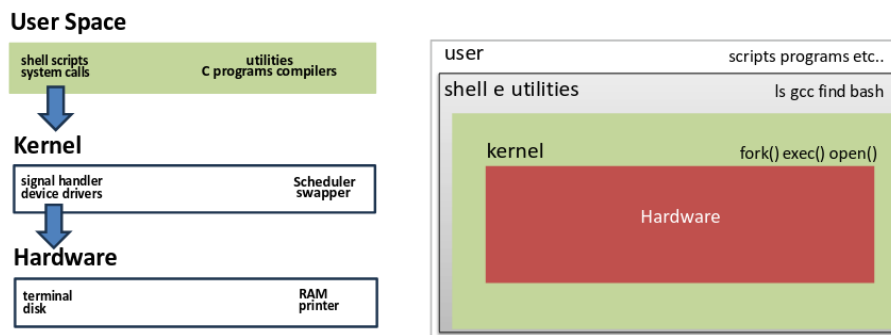
- Programmi
- Librerie
- System call (per comunicare con il Kernel)
- Una *shell* (interprete dei comandi in `exec/fork`) per gestire I/O e pipeline.

3.1 Unix

Uniplexed Information and Computing Service nasce fra gli anni '60 e '70 come sistema operativo proprietario

- multiutente
- multitasking
- portatile (essendo scritto in C poteva girare su più macchine)
- minimale
- TCP/IP (dal 1984) permette di comunicare fra più macchine diverse

Negli anni, il codice sorgente di UNIX è stato concesso in licenza a varie università e aziende. Ognuna ha creato una propria “versione” di Unix, adattata alle proprie esigenze hardware o commerciali (Solaris di Oracle, BSD Unix, ecc). Ad oggi solo i sistemi certificati (come MacOS) sotto il marchio registrato di The Open Group possono definirsi, legalmente, Unix.



Ad oggi troviamo molti *Unix-like*, ovvero SO che si basano su vecchi ceppi di Unix o si ispirano fortemente

- BSD: FreeBSD, OpenBDS, NetBSD (non sono certificati come Unix ma derivano storicamente da Unix)

- MacOS / IOS (basati su BSD + kernel XNU)
XNU = "X is not Unix" e' un kernel sviluppato da Apple ibrido, ovvero BSD + Mach (processi, thread, memoria, ecc) + Driver (proprietary)
- Linux: Linux e' un kernel libero compatibile con Unix distribuito con licenza GPL, partendo da zero, cioè senza usare alcuna riga di codice Unix. Si basa sugli strumenti GNU. Si occupa di gestire:
 - Processi
 - Memoria
 - Driver
 - Filesystem
 - Networking

Tutti 3 gli OS qui sopra si basano sullo standard POSIX (Portable Operating System Interface, with an X for Unix), è una famiglia di standard sviluppata dall'IEEE a partire dagli anni '80 (serie IEEE 1003), in collaborazione con ISO. Uno standard del genere ci permette di avere software portatili tra i sistemi operativi Unix-like, attraverso API, comandi e comportamenti comuni.

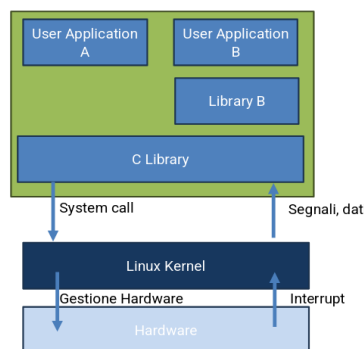
3.2 Cosa definisce POSIX?

- System call, API: per la gestione dei processi come "fork(), exec(), wait(), ecc", gestione dei file "open(), read(), write()", gestione della IPC per pipe, semafori, memoria condivisa, "kill(), signal()"
- Comandi di base: "ls, cp, mv, ecc"
- Shell

3.3 GNU

Il progetto GNU (Richard Stallman, Free Software Foundation, dal 1983) voleva creare un sistema operativo libero in stile Unix. GNU sviluppo': librerie (glibc), compilatore (gcc), shell (bash), strumenti base. Mancava ancora però un kernel! Si è fatta una prova con "GNU Hurd" ma si decise di andare con il kernel linux. Definiamo **GNU/Linux** il sistema operativo completo.

Ovviamente essendo un modello libero, Unix-like con standard POSIX e molto flessibile in breve tempo si diffusero diverse **distribuzioni**.



Il kernel deve occuparsi delle risorse hw (CPU, memoria, IO), deve fornire delle API portatili e standardizzate tra le architetture, gestire accessi concorrenti.

Fra lo *user space* e il kernel troviamo le system call, ovvero un set di comandi del kernel (circa

400 ad oggi). Tutte le system call le troviamo nella libreria C, ma non le chiamiamo mai direttamente ma convochiamo le rispettive versione della funzione in C.

Il filesystem segue convenzioni come FHS (filesystem hierarchy standard)

- / (root), /bin, /usr, /lib, /etc, /dev, /proc, /sys.
- /dev (dispositivi come file speciali)
- /proc, /sys (informazioni del kernel e device in modo file-like)

Il C rimane essenziale dove contano controllo e prevedibilità:

- kernel, driver, librerie di base, sistemi embedded ed edge.
- Senza garbage collector e con puntatori espliciti, rende chiari i rapporti tra sicurezza e prestazioni.

Quindi possiamo definire il C come il linguaggio privilegiato quando servono efficienza, interoperabilità e un legame diretto con l'hardware.